BBM-382 – SOFTWARE ENGINEERING

SPRING 2025

Lecture 8

Prof.Dr. Ayça KOLUKISA TARHAN
Assoc.Prof.Dr. Ebru Gökalp
Asst.Prof.Dr. Tuğba ERDOĞAN

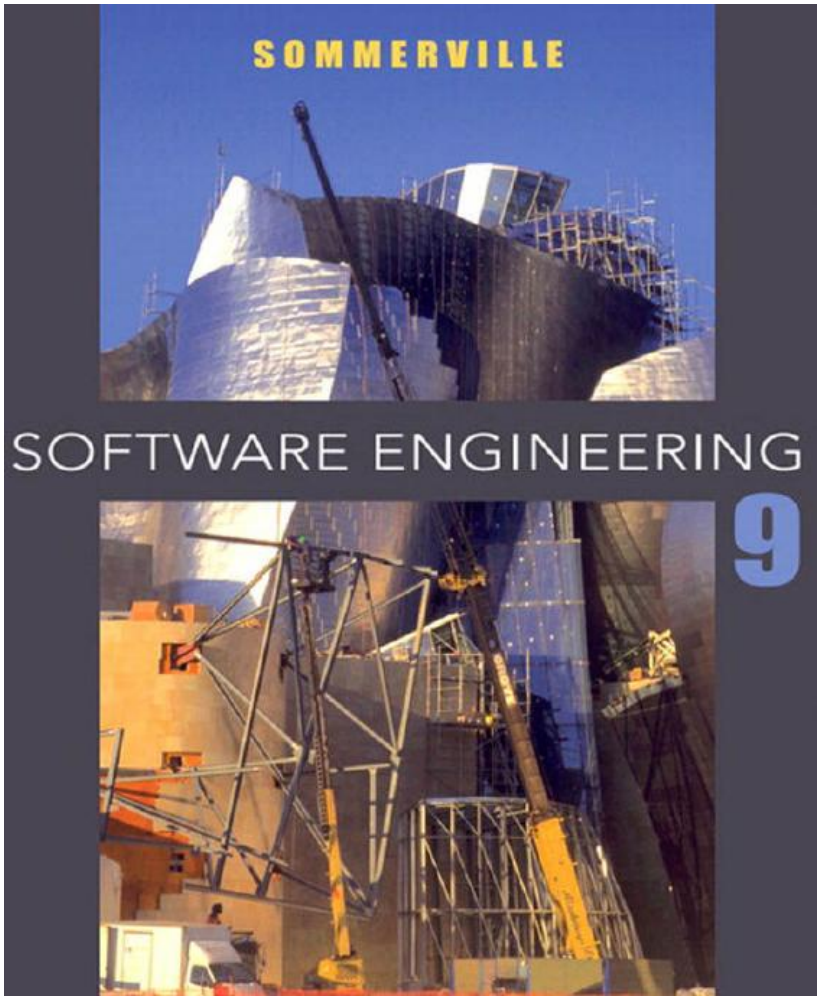
Reference:

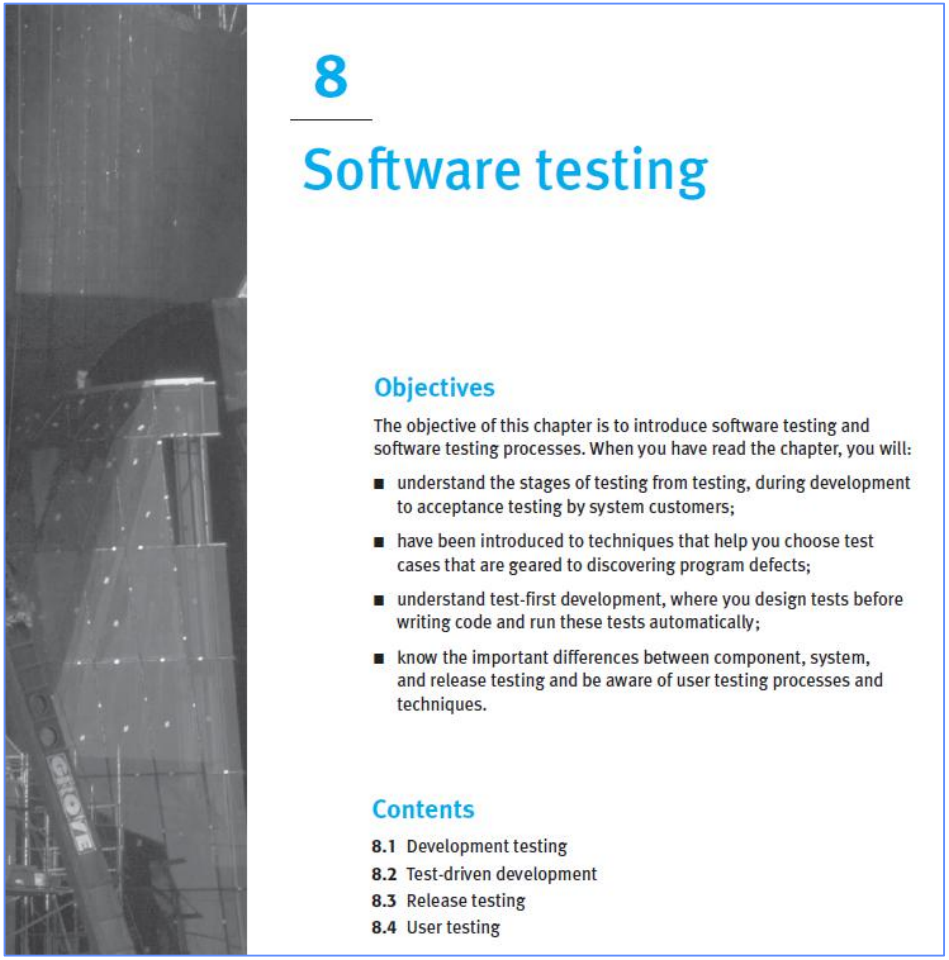
Textbook Ch8-Software Testing

Lecture Notes by Vahid Garousi

From the book

- Read it online!





8

Software testing

Objectives

The objective of this chapter is to introduce software testing and software testing processes. When you have read the chapter, you will:

- understand the stages of testing from testing, during development to acceptance testing by system customers;
- have been introduced to techniques that help you choose test cases that are geared to discovering program defects;
- understand test-first development, where you design tests before writing code and run these tests automatically;
- know the important differences between component, system, and release testing and be aware of user testing processes and techniques.

Contents

8.1 Development testing
8.2 Test-driven development
8.3 Release testing
8.4 User testing

Need for SW testing-Examples of SW Failures

ComputerWeekly.com

News | IT Management | Industry Sectors | Technology Topics | Blogs | Multimed

NEWS

Lack of software testing to blame for Terminal 5 fiasco, BA executive tells MPs

Home > Applications > News

Applications

Jaguar recalls 18,000 cars over cruise control software fault

Car system upgrade needed, but no hardware affected

By Leo King | Computerworld UK | Published 10:23, 24 October 11

Examples of Software Failures

FOX BUSINESS
THE POWER TO PROSPER™

WHAT'S HOT Retirement • European

Home | Video | News | Investing | Technology | Personal Finance | Home Office | Travel

Honda Recalls 2.5M Vehicles Due to Software Problem

Published August 05, 2011 / Reuters

Print

Email

Honda Motor Co Ltd will recall 2.49 million cars, small SUVs and minivans worldwide, including its popular Accord sedan, to repair a software problem that could damage the automatic transmission.

Toyota software bugs unlike those in flaky PCs

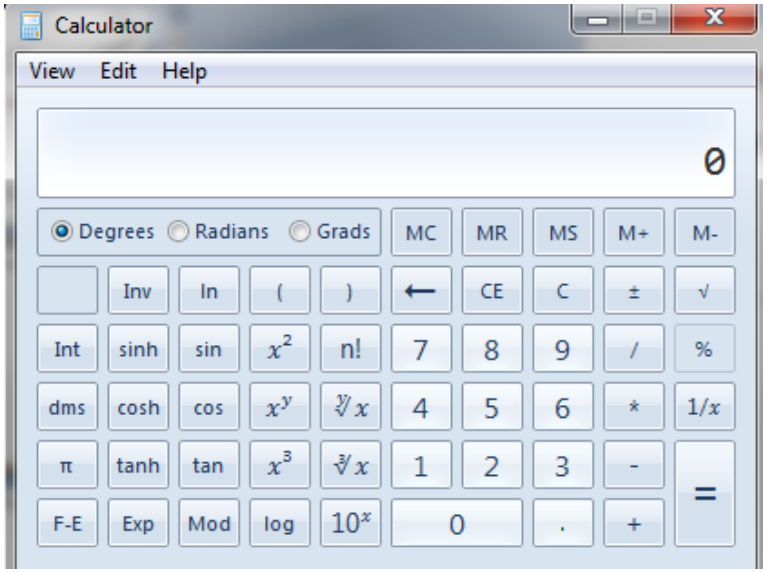
by Brooke Crothers | February 18, 2010 8:39 AM PST

Follow

The electronic issues dogging the Toyota Prius signal that cars are increasingly susceptible to software bugs. Cars, however, are not PCs on wheels and have a different set of problems than that crash-prone computer on your desktop, according to an expert at Edmunds.com.

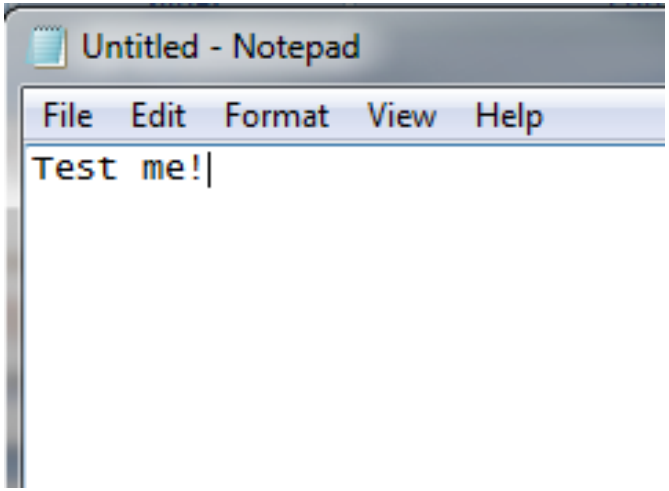
Brain storming: How will you test the Win Calculator?

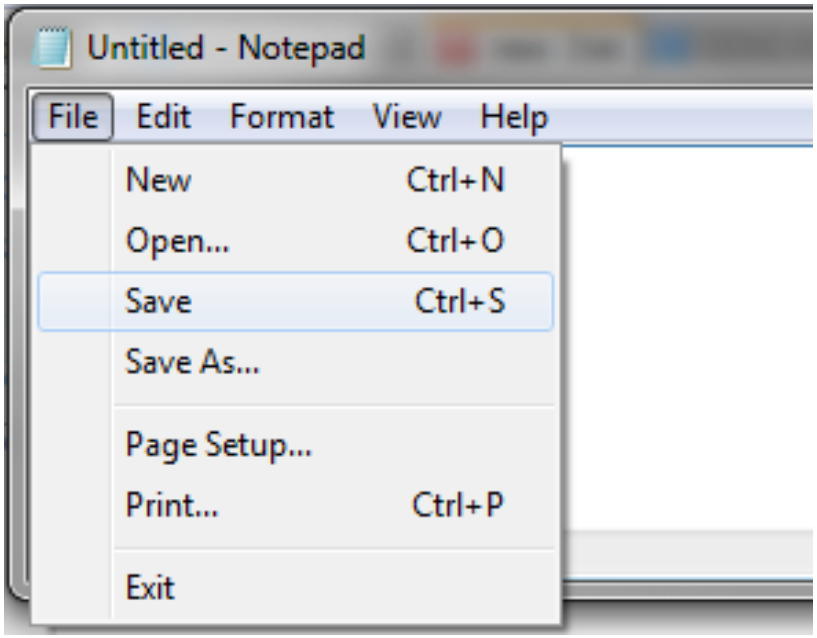
- Test plan
- How many inputs (test cases)?
- So MANY possibilities
- How do we know if we have tested the system “enough”?
- How do we know if the outputs are correct!?
- If the system changes, shall we test everything AGAIN?
- Tedious!

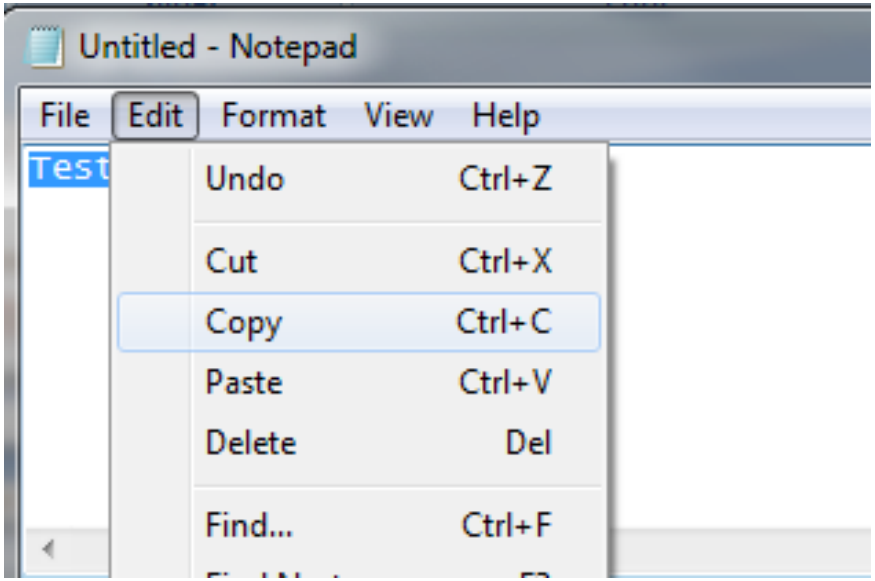


Brain storming: How will you test the Notepad?

- Test-case Design: Exploratory
- Test-case Design: Criteria-based
- Test Automation
- Test Evaluation







Introduction to Software Testing- Outline

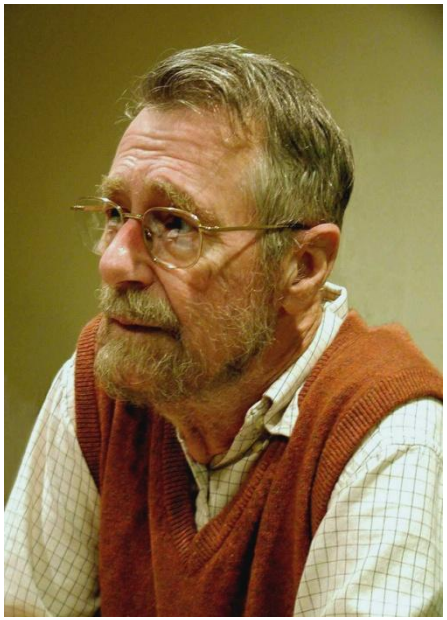
- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Definitions

- **SW Project Management:** The discipline of managing projects to achieve quality within time constraints and budget.
- **SW Quality (Engineering):** The discipline of specifying, assuring, monitoring, and controlling the quality of software products.
- **SW Verification:** Did we build the product right?
  - Ensures that the product satisfies or matches the original design (low-level checking).
  - This is usually done through white-box testing.
- **SW Validation:** Did we built the right product?
  - Checks that the product design satisfies or fits the intended usage (high-level checking).
  - This is usually done through black-box testing.

Definitions

- **SW Testing:** Techniques to execute programs with the intent of finding as many defects as possible and/or gaining sufficient confidence in the software system under test.
  - ***"Program testing can show the presence of bugs, but never their absence"*** **(Edsger Dijkstra, 1930-2002)**



- **SW Inspections:** Techniques aimed at *systematically* verifying software artifacts (design documents, code, tests, etc) with the intent of finding as many defects as possible, as early as possible.

What SW testing is and is NOT

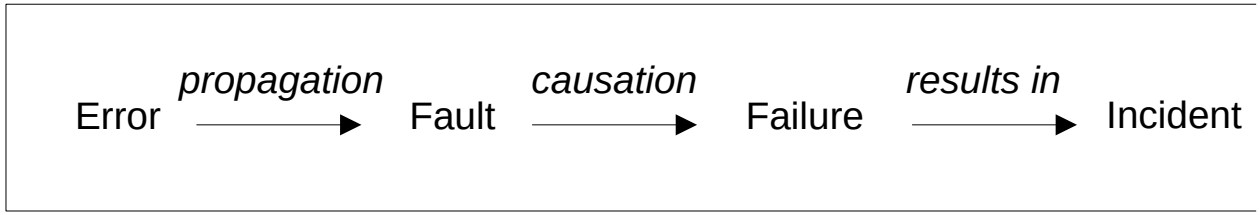
- Testing is to execute a program with the intent of finding as many defects as possible and/or gaining sufficient confidence in the software system under test
- Testing is not cut-and-fit improvement of a user interface or requirements elicitation by prototyping.
- Testing is not the verification of an analysis or design model by syntax checkers or simulation.
- Testing is not the scrutiny of documentation or code by humans in inspections, reviews, or walkthroughs.
- Testing is not static analysis of code using a language translator or code checker.
- Testing is not the use of dynamic analyzers to identify memory leaks or similar problems.
- Testing is not debugging, although successful testing should lead to debugging.

Goals and Limitations of Testing

- Dijkstra, 1972:
  - *"Program testing can be used to show the presence of bugs, but never to show their absence"*
- Thus:
  - No absolute certainty can be gained from testing
  - Testing should be integrated with other SW quality activities, e.g., code reviews and inspections
  - Main goal of testing: demonstrate the software can be depended upon, i.e., *sufficient dependability*
  - We can only gain <u>sufficient</u> confidence in the System Under Test (SUT), but not full confidence!

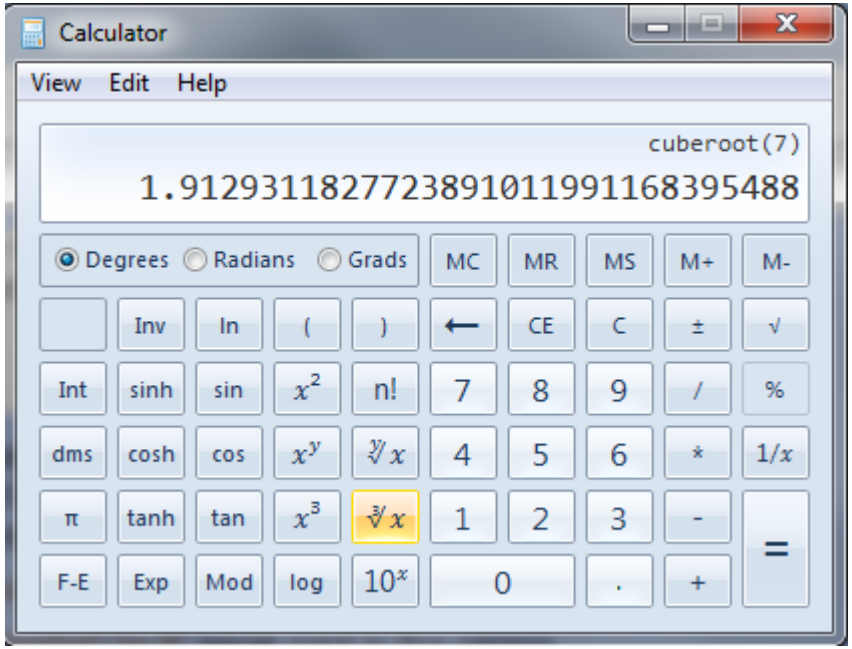
Basic Testing Definitions

- **Error:** the very root cause. Errors are committed by people
- **Fault:** A fault is the result of a human error in the software documentation, code, etc.
- **Failure:** A failure occurs when a fault executes
- (Many people use the above three terms inter-changeably. We should not do so!)
- **Incident:** Consequences of failures – Failure occurrence may or may not be apparent to the user
- **Defect:** any of the above, usually a fault, a.k.a. bug
  - E.g., people say: “my code has a bug!”
- The fundamental chain of SW dependability threats:

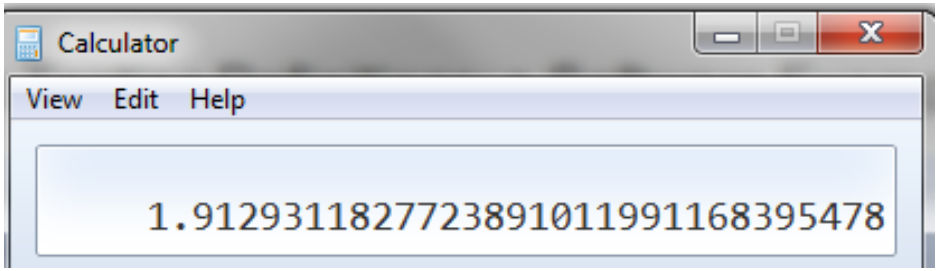


Basic Testing Definitions: a Software Example

- Cube root of 7 in Calculator App:
- The correct expected value: →



- A human SW tester runs the app manually and gets:



- She does not notice the fault in this case.
- Let's discuss the possible errors, faults and failures in this case.

Introduction to Software Testing- Outline

- Definitions
- **Why is SW testing important?**
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Why is SW testing important?

- A study by (the American) NIST in 2002: The annual national cost of inadequate testing is as much as $59 Billion US!
  - The report is titled: “The Economic Impacts of Inadequate Infrastructure for Software Testing”
  - http://www.nist.gov/public_affairs/releases/n02-10.htm

Software Errors Cost U.S. Economy $59.5 Billion Annually

NIST Assesses Technical Needs of Industry to Improve Software-Testing

www.turkishtestingboard.org

TTB

Turkish Testing Board

Your gateway to global Software Testing community

ISTQB
International Software Testing Qualifications Board
www.istqb.org

Anasayfa | Hakkımızda | Sertifikasyon | Eğitim | Sertifika Kazananlar | TestIstanbul Konferansları | TSQR | Haberler | İletişim | English

ISTQB Büyümeye Devam Ediyor

TTB'nin yönetiminde yer aldığı dünyanın en büyük Yazılım Test ve Kalite Organizasyonu ISTQB 70 ülkede 240.000'den fazla bilişim profesyonelini sertifikalandırmıştır.

daha fazlası için

Uluslararası Akredite Eğitim Şirketleri

ISTQB Uluslararası Sertifikalı Yazılım Test Uzmanı Eğitimleri ISTQB tarafından akredite edilmiş eğitim kurumları tarafından verilmektedir. ISTQB tarafından onaylanmış "Accredited Training Provider" logolu eğitim kurumları:

tested by keytorc "excellence in software testing"

ISTQB Accredited Training Provider

Dünya Yazılım Test ve Kalite Sektörü İstanbul'da Buluşmaya Devam Ediyor...

TEST ISTANBUL

Future of Testing:
New Techniques and Methodologies
23-24 May,2013

Software Test Engineers at Microsoft

proquest.safaribooksonline.com.ezproxy.lib.ucalgary.ca/book/software-engineering-and-development/software-testing/9780735624252/software-test-en

Safari Books Online

HOME | Quick Links

[Start a New Search] | Entire Site | SEARCH | ADVANCED SEARCH | Sign Out & Clear Sessi

This Book

How We Test Software at Microsoft®

Search

Contents

- Table of Contents
- Copyright
- Acknowledgments
- Introduction
- Pt. I. About Microsoft
  - Ch. 1. Software Engineering at Microsoft
  - Ch. 2. Software Test Engineers at Microsoft
    - What's in a Name?
    - Testers at Microsoft Have Not Always Been SDETs
    - I Need More Testers and I

< Return to Search Results

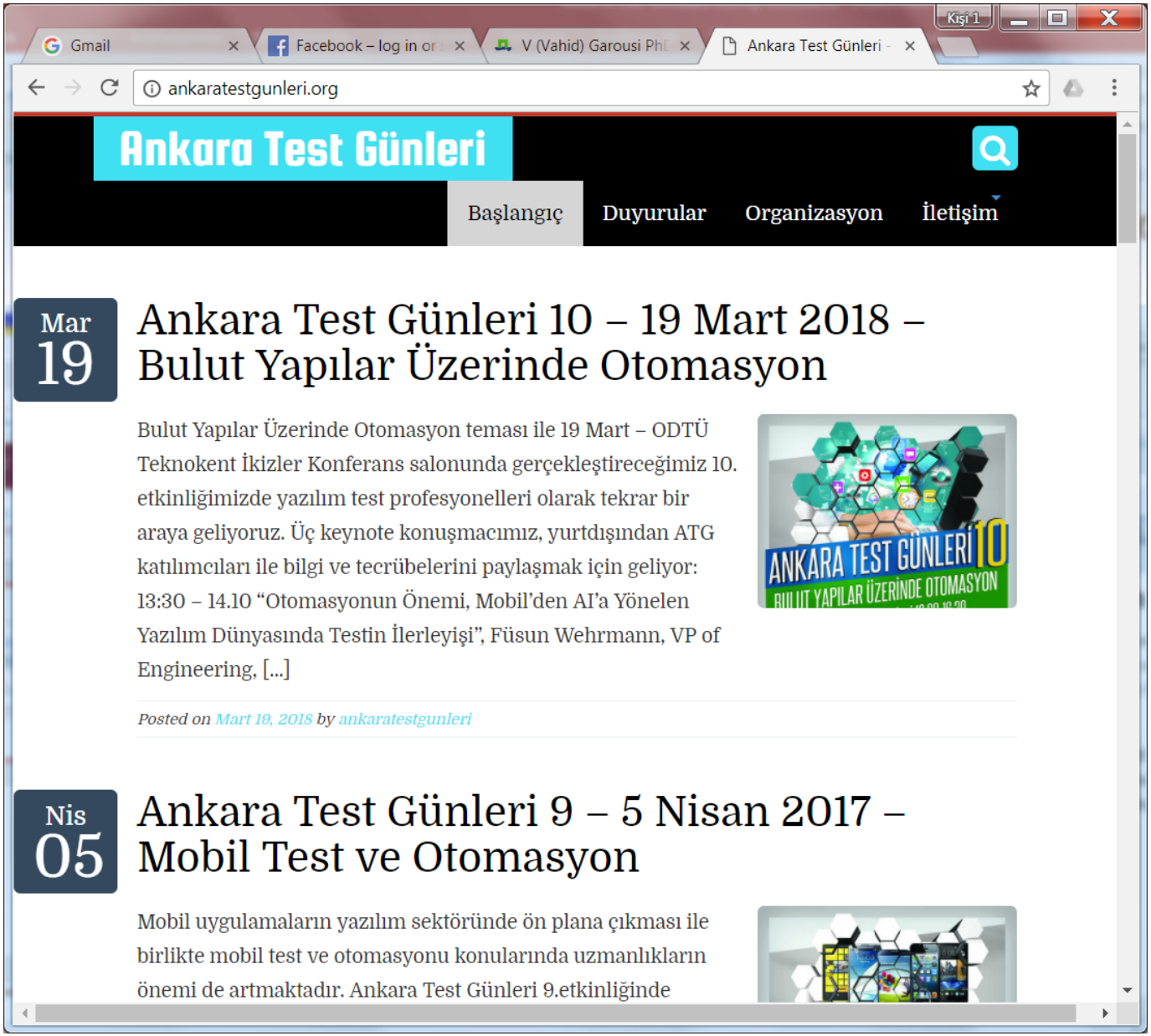
Chapter 2. Software Test Engineers at Mi... > I Need More Testers and I Need Them ... - Pg. 27

I Need More Testers and I Need Them Now!

Microsoft continues to increase its engineering workforce every year. Test alone adds approximately 500 new positions a year. We have a nearly even split in hiring testers from other companies and computer science graduates straight from colleges both in the United States and in leading universities around the world.

A great SDET candidate possesses that tester DNA covered earlier in this chapter. We also look for some very specific skills or, in HR lingo, "competencies." To prevent this section from reading too much like an HR manual, I just briefly cover what a *competency* is and how it relates to tester DNA.

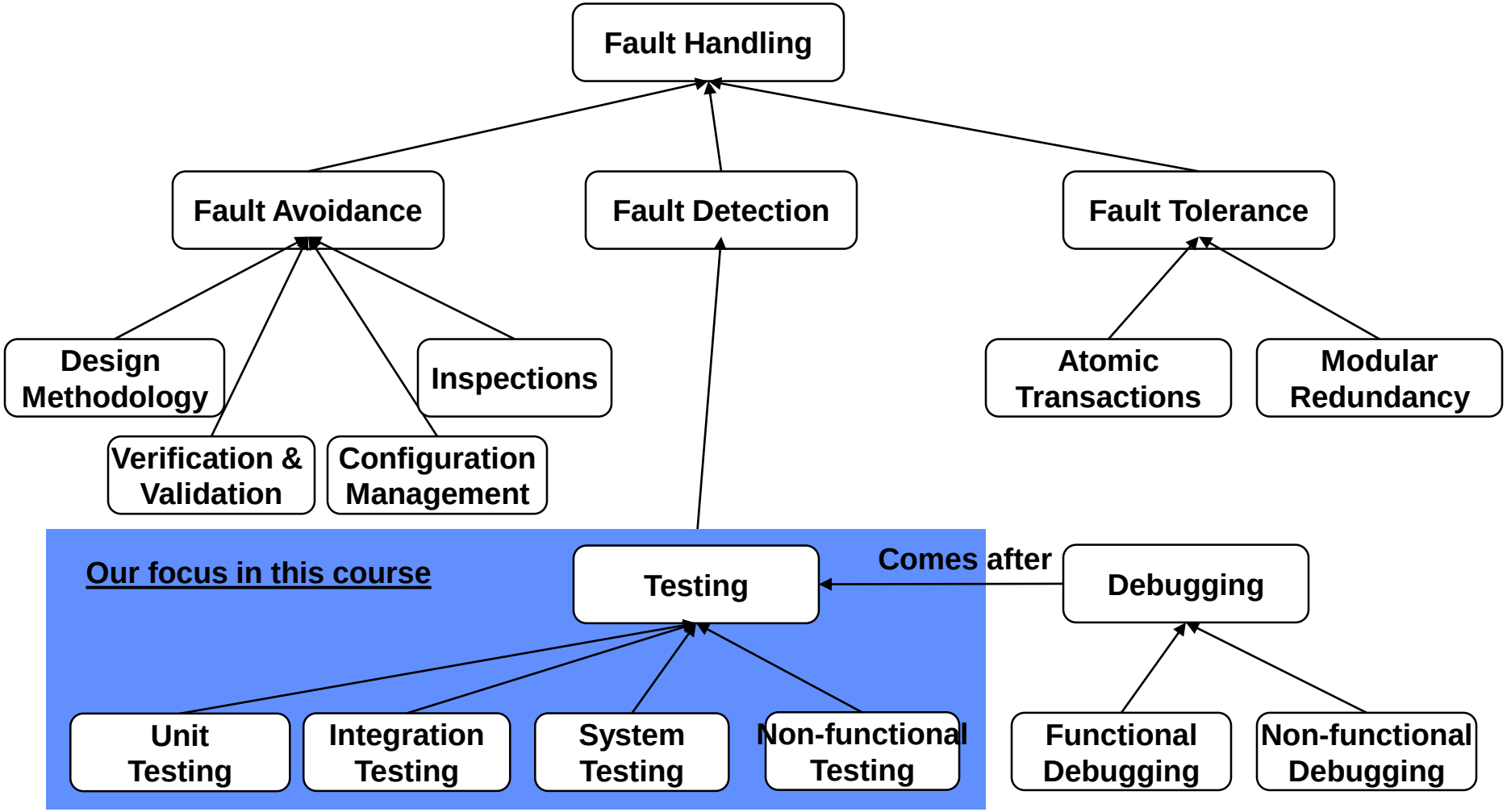
Ankara Test Günleri



Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- **Dealing with SW Faults**
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Dealing with SW Faults / Software Quality Engineering (SQE)

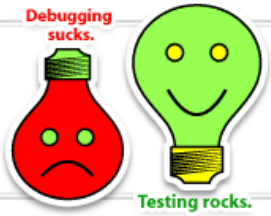


Motto from Google!

googletesting.blogspot.com

Google Testing Blog

If it ain't broke, you're not trying hard enough



RPF: Google's Record Playback Framework

Thursday, November 17, 2011 5:26 AM

By Jason Arbon

At GTAC, folks asked how well the Record/Playback (RPF) works in the Browser Integrated Test Environment (BITE). We were originally skeptical ourselves, but figured somebody should try. Here is some anecdotal data and some background on how we started measuring the quality of RPF.

The idea is to just let users use the application in the browser, record their actions, and save them as a javascript to play back as a regression test or repro later. Like most test tools, especially code generating ones, it works most of the time but its not perfect. Po Hu had an early version working, and decided to test this out on a real world product. Po, the developer of RPF, worked with the chrome web store team to see how an early version would work for them. Why chrome web store? It is a website with lots of data-driven UX, authentication, file upload, and it was changing all the time and breaking existing Selenium scripts: a pretty hard web testing problem, only targeted the chrome browser, and most importantly they were sitting 20 feet from

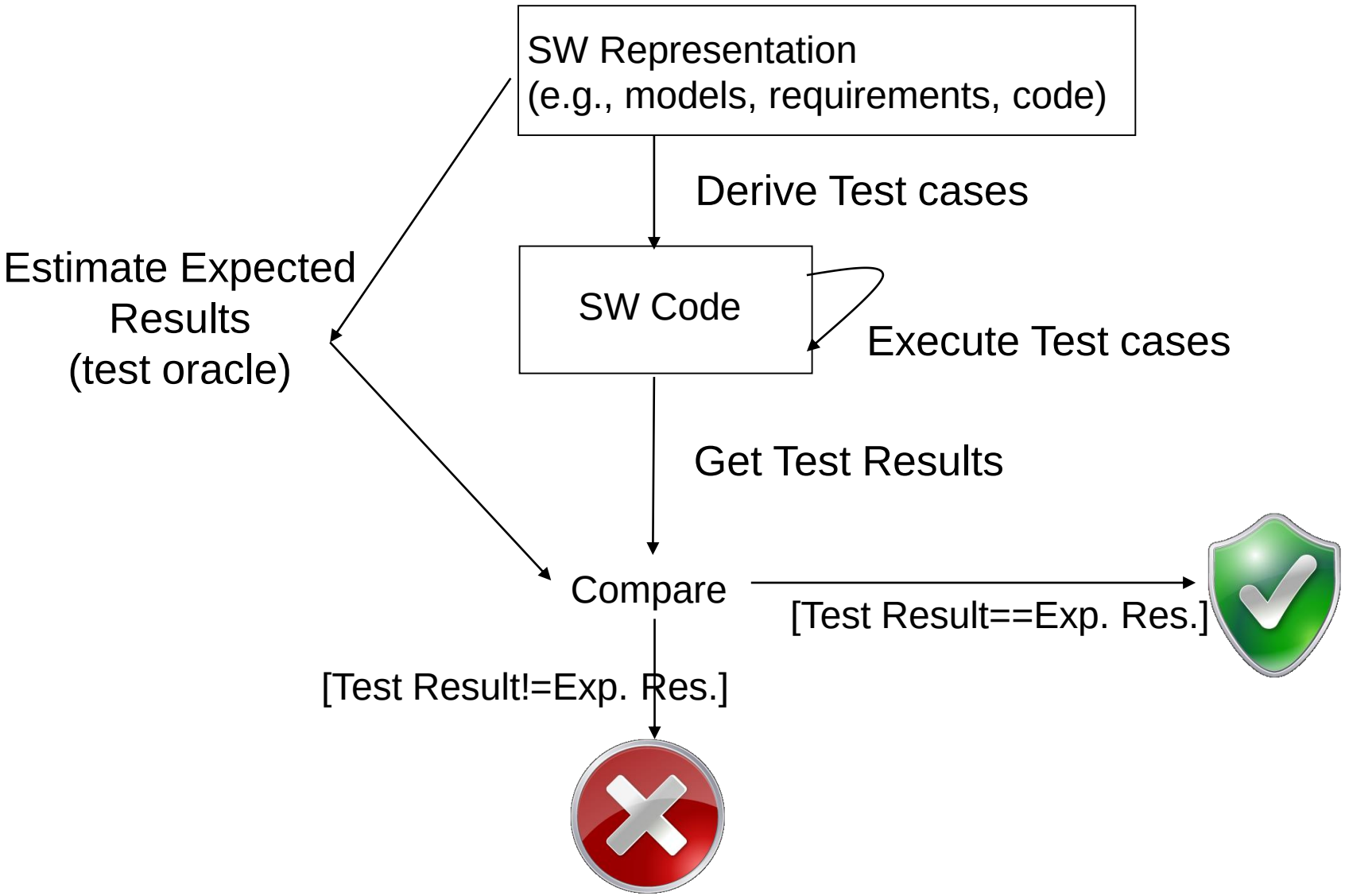
Search ×

powered by Google™

Tags

- Whittaker (57)
- GTAC (33)
- Copeland (26)
- Conferences (14)
- TotT (12)
- Misko (8)
- chromeos (2)
- test (2)
- testing (2)
- Alberto has gone off the deep end (1)
- Coverage (1)

Testing Process Overview

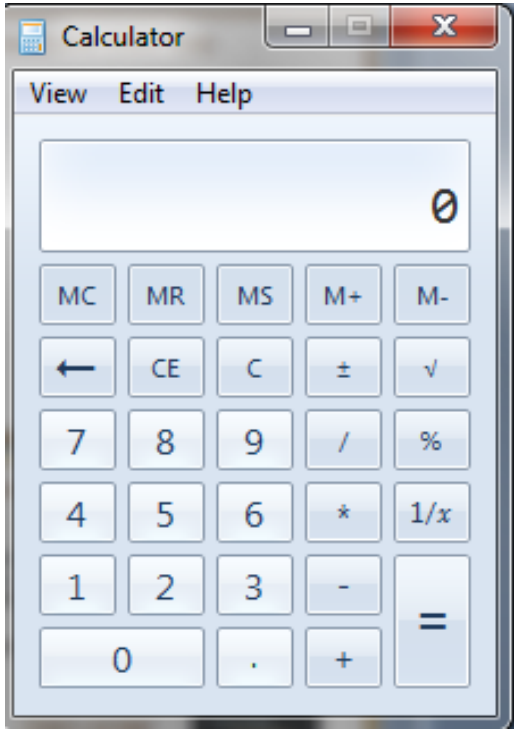


Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- **What is a Test Case and Test Suite (Test Set)?**
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

What is a Test Case and Test Suite (Test Set)?

- A **test case** is set of inputs and the expected outputs for a unit/module/system under test
- A **test suite (test set)** is set of test cases (at least one)
- Without the expected outputs, a test case is NOT complete.
- For Example, for Calculator:

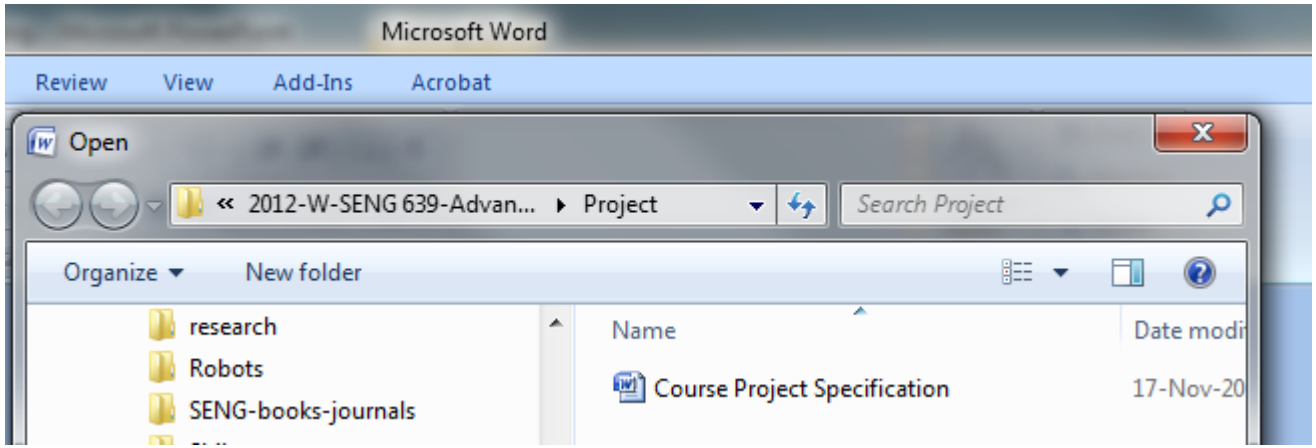


TC#	Unit under test	Input 1	Input 2	Expected output
1	+	5	6	11
2	+	16	0	16
3	/	5	2	2.5
4	/	4	0	Cannot divide
	…			
	So many more test cases!			

Test Case and Test Suite (Test Set)

- The above definitions hold for test cases in any level: unit, integration or system testing
- Example, for Microsoft Word:

TC#	Feature under test	Inputs	Expected output
1	Open	Valid DOCX file	File opened properly
2	Open	Invalid DOCX file	Error message
3	Open	RTF file	File opened properly
	…		
	So many more test cases!		



Test Case and Test Suite (Test Set)

- The definitions hold for test cases in any level, unit, integration or system testing

```
int abs(int x){
    if (x>0)
        return x;
    else
        return -x;
}
```

- Example, for the `abs` function:

TC#	Unit (method) under test	Inputs	Expected output
1	abs	A +ve, 3	3
2	abs	A –ve, -5	5
	Are two test cases enough? What about 0?		

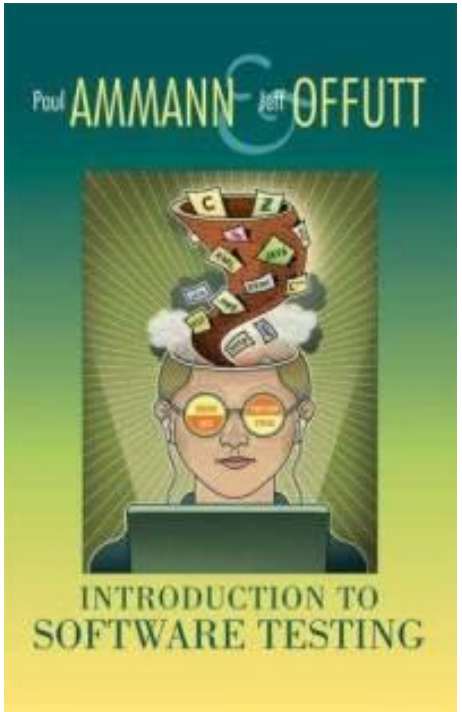
- The art and science of deriving efficient but effective number of test cases is called “test-case design”.
- “Less is more!” - Less test cases, but maximizing the chance of covering faults.

Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- **Types of Test Activities**
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Types of Test Activities

- **Test-case Design:** Exploratory (Human-based)
  - Design test values based on domain knowledge of the program and human knowledge of testing, exploratory testing
- **Test-case Design:** Criteria-based
  - Design test values to satisfy coverage criteria, covering all lines of code
- **Test Automation**
  - Embed test values into executable scripts
- **Test Execution**
  - Run tests on the software and record the results
- **Test Evaluation**
  - Evaluate results of testing, report to developers



Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- **Types of Testing**
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Types of Testing

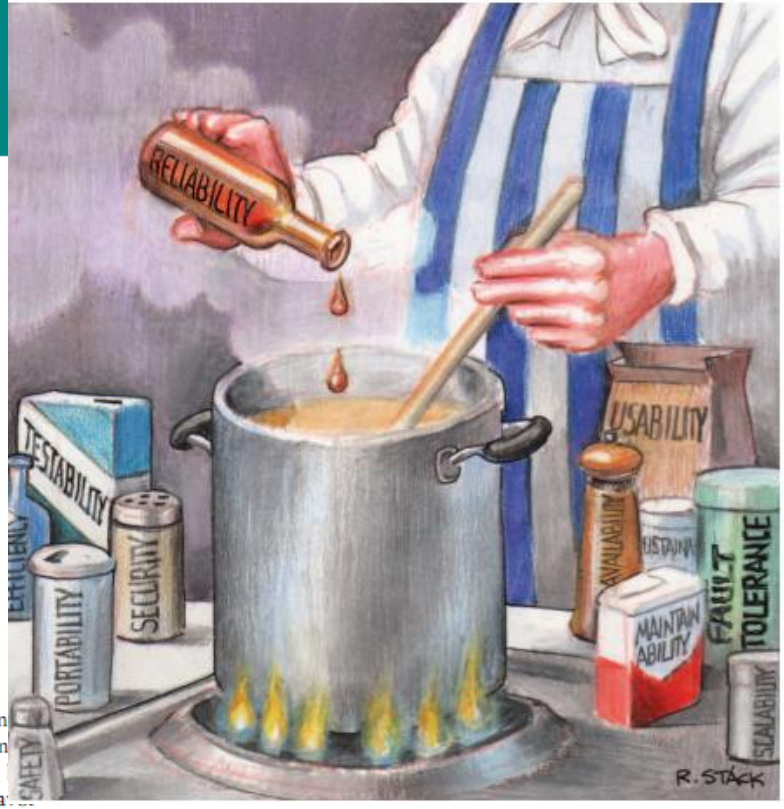
- **Functional Testing: testing functional requirements**
  - Functional testing is checking correct functionality of a system

- **Non-functional Testing: testing non-functional requirements**
  - **Performance testing**
  - Load testing
  - Endurance testing
  - Localization/Internationalization testing
  - Recovery testing
  - Security testing
  - Stress testing
  - Installation testing
  - Compatibility testing
  - Usability testing
  - Scalability testing
  - Reliability testing
  - …ilities!

Editors: Nancy Eickelmann ■ Motorola ■ nancy.eickelmann@motorola.com
Jane Huffman Hayes ■ University of Kentucky ■ hayes@cs.uky.edu

Software's Secret Sauce: The "-ilities"

Jeffrey Voas



If beauty is in the eye of the beholder, then quality must be as well. We live in a world where beauty to one is a complete turnoff to another. Software quality is no different. We have the developer's perspective, the end users' perspective, the testers' perspective, and so forth. So, for instance, if the software complies with the requirements, is that quality software? Many would say yes, but then, what if the software isn't fit for any reasonable purpose? Or is quality software simply software that was developed according to particular standards and regulations?

As you can see, meeting the requirements might be different from being fit for a purpose, which can also be different from complying with rules and regulations on how to develop and deploy the software. Yet we can think of all three perspectives as ways to determine how to judge and assess software quality.

To everything there's a season

when added. To use an analogy, an -ility in application or system is like a condiment on entrée: not valuable as a stand-alone item capable of significantly enhancing the fla when added properly. Examples of these flavor-enhancing software -ilities include maintainability, reliability, usability, efficiency, adaptability, availability, security, portability, scalability, safety, fault tolerance, testability, usability, reusability, and sustainability.

So, if we look at the issue of the software meeting its requirements, and if those requirements are solely functional and prescribe no -ilities, then clearly the software can meet the requirements but could be unable to fulfill any reasonable purpose. Furthermore, the -ilities will unlikely be associated in any way with how the software is developed, because development standards rarely drill down low enough to provide the guidance needed to attain particular degrees of any -ility.

Measure for measurement

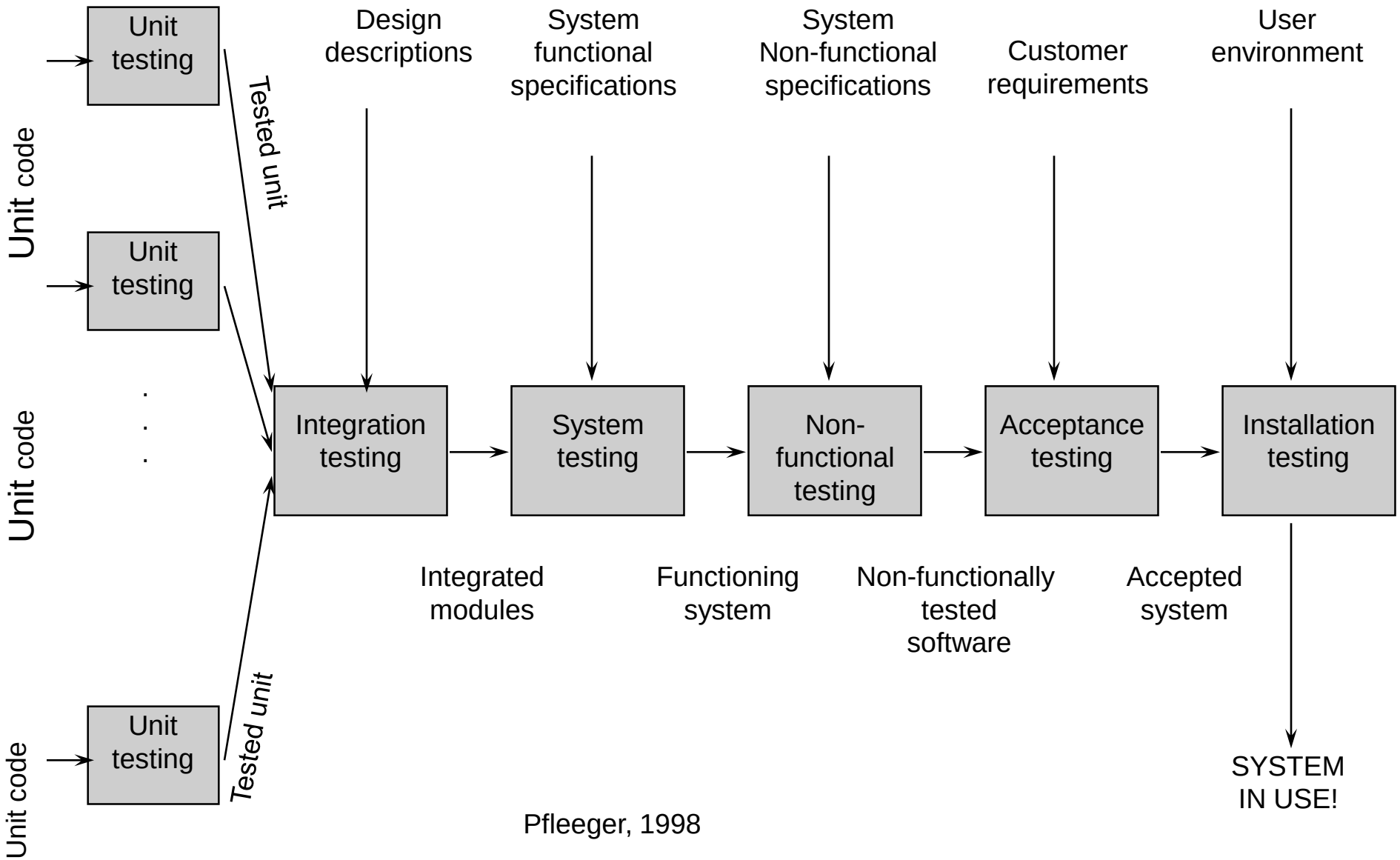
Note that degree is important. For example, a little salt on a steak usually tastes great.

Test Levels / Test Organization

- Since we can have defects in different level and stages of the SW process, we need **testing at different levels and stages**
  - Unit testing (also called module or component testing)
  - Integration testing
  - System testing
  - Non-functional (e.g., performance) testing
  - Acceptance testing
  - Installation testing
  - …

- One typical test process comes next…

Test Organization (just one typical test process)



Pfleeger, 1998

Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- **Testing can be Manual or Automated**
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

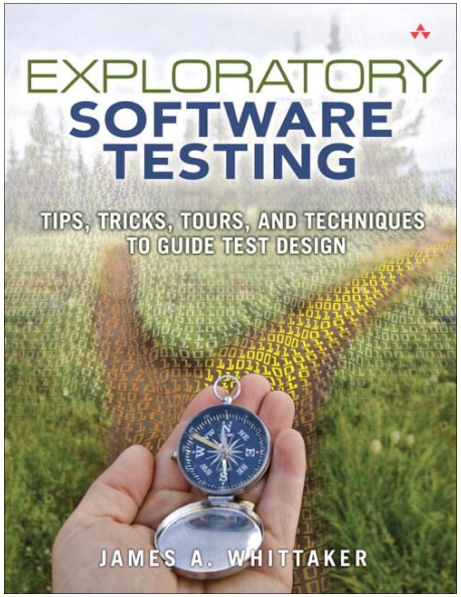
Testing can be Manual or Automated

Manual Testing

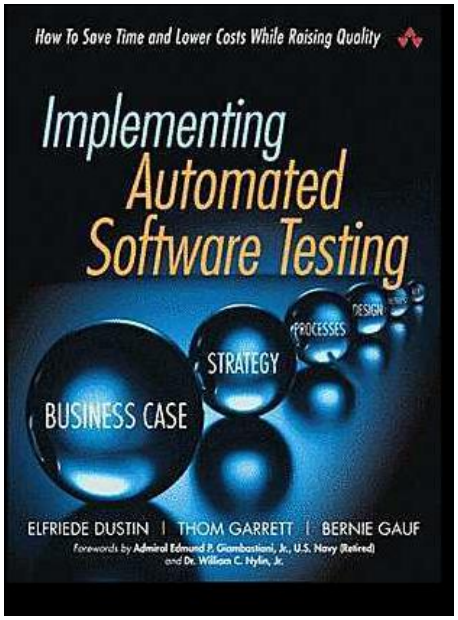
- Clever test-case design
- Interaction with system inspiration for new tests
- Human oracle
- Single test case execution
- Limited data

Automated Testing

- Specs, models, & code used to derive test cases
- Automated oracle needed
- Test execution easily repeatable
- Massive input data possible



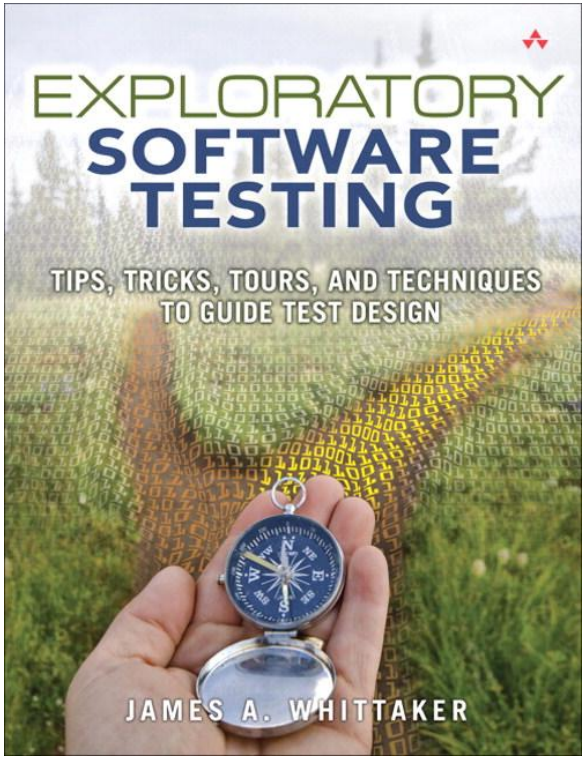
Clearly,
we need both
and
both have
advantages/
disadvantages



Manual Testing: Exploratory Software Testing

- **Human tester**
  - using brain, and fingers
  - to create scenarios that
  - will cause software either to fail or to fulfill its mission

- **Take advantage of human cleverness**
  - No scripts: Exploratory Testing
  - Or manual scripted testing
  - Record test findings as you go

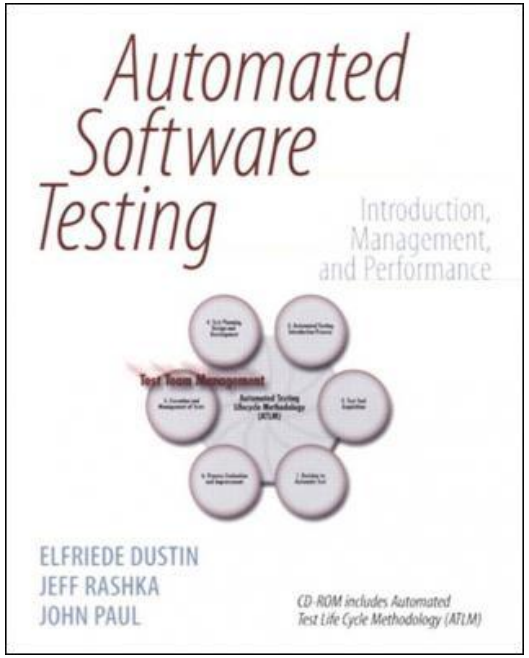


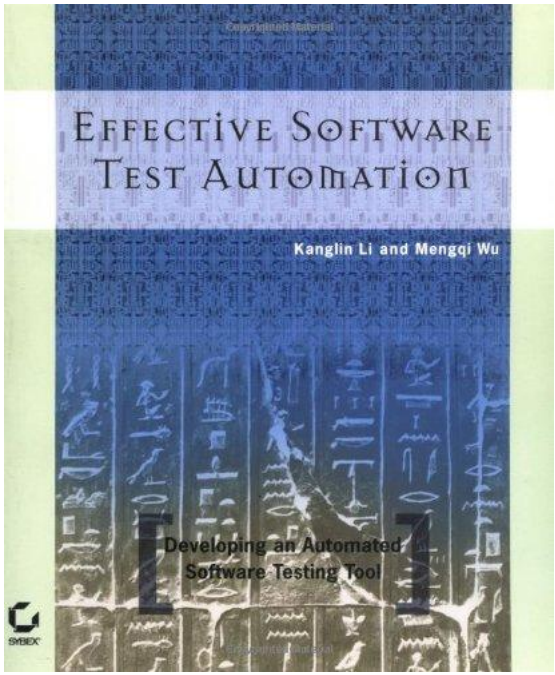
The Problem with Manual Testing

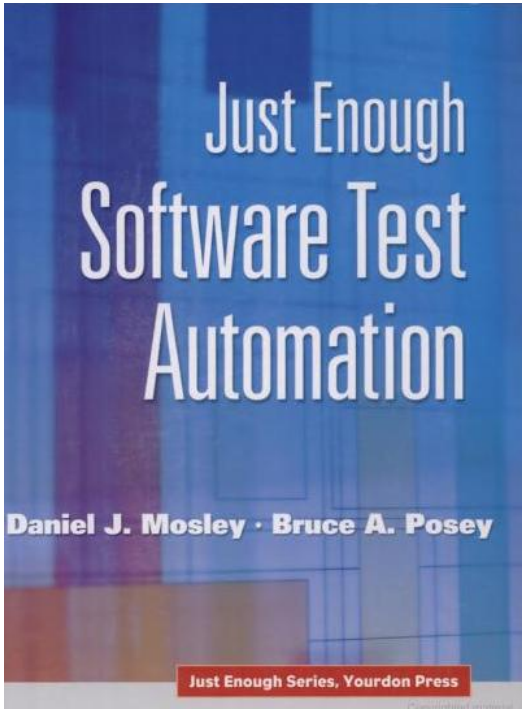
- Not very repeatable
  - Either cannot remember exactly how it was last tested (what input value did I use a month ago to fail the software?)
  - Or, too expensive to document detailed procedures. Perhaps you have seen “thick” manual test scripts still some companies use.
- Very effort intensive
  - How long will it take to re-run all the tests?
  - How often will you actually do it?

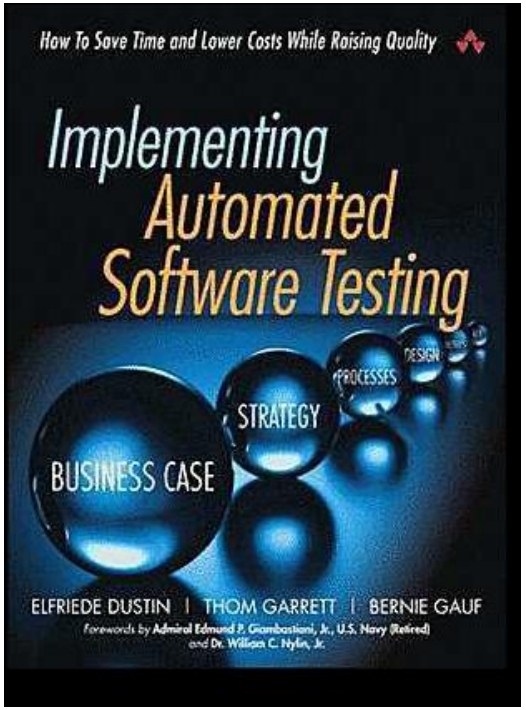
Automated testing

- Using a test software (tool) to test another software
- Developing test code (script) to test other piece of code (application code)
- How much do you know about it?









Why Automate Tests?

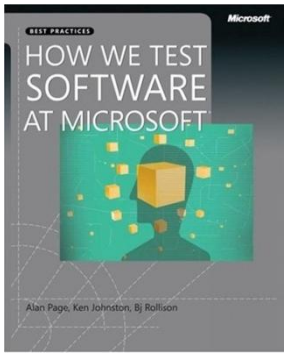
- It gives us rapid feedback (pass/fail)!
- Repeatable
- Much faster than human testers, but needs up-front investment
- How do we know where defects are?
  - We remember where we put them!
  - Defect (fault) localizations

```
int abs(int x){
    if (x>0)
        return x;
    else
        return -x;
}

int test_abs_positive(){
    if (abs(5)==5)
        return 1;
    else
        return 0;
}

int test_abs_negative(){
    if (abs(-4)==4)
        return 1;
    else
        return 0;
}
```

Increasing size and complexity for automated test systems



Note There were more than a million test cases written for Microsoft Office 2007.

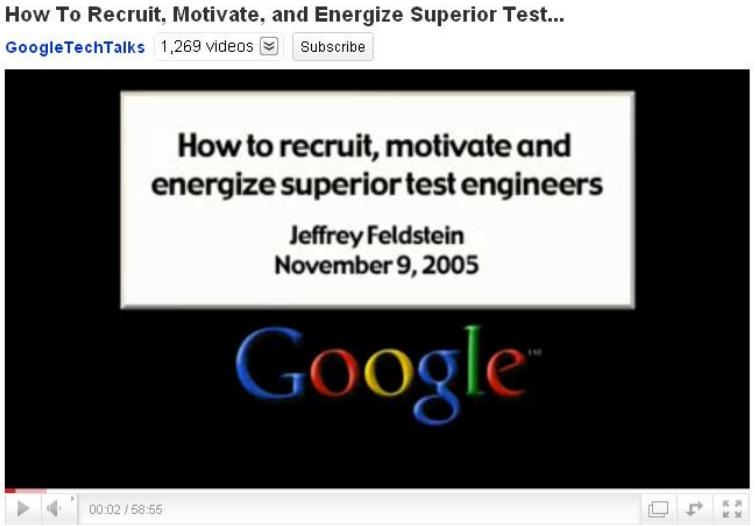
- "We designed a test system that probably is as complicated as the system itself."

Jeff Feldstein, Manager, Software Development

Cisco Systems, San Jose, California

Google Tech Talks, November 9, 2005

(a very good talk)



Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automated
- What is a good test case?
- **Exhaustive (Complete) Testing**
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Exhaustive (Complete) Testing

- Exhaustive testing, i.e., testing a software system using all the possible inputs, is most of the time impossible.

- Examples:
  - **A program that computes the factorial function (n!=n.(n-1).(n-2)…1)**
    - **Exhaustive testing = running the program with 0, 1, 2, …, 100, … as an input!**
  - **A compiler (e.g., javac)**
    - **Exhaustive testing = running the (Java) compiler with any possible (Java) program (i.e., source code)**

- Techniques used to reduce the number of inputs
  - Testing criteria group input elements into (equivalence) classes
  - One input is selected in each class (notion of **test data coverage**)

Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- **Test Adequacy and Coverage: When can we say testing is enough?**
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels

Test Adequacy: When can we say testing is enough?

Functional (black-box)

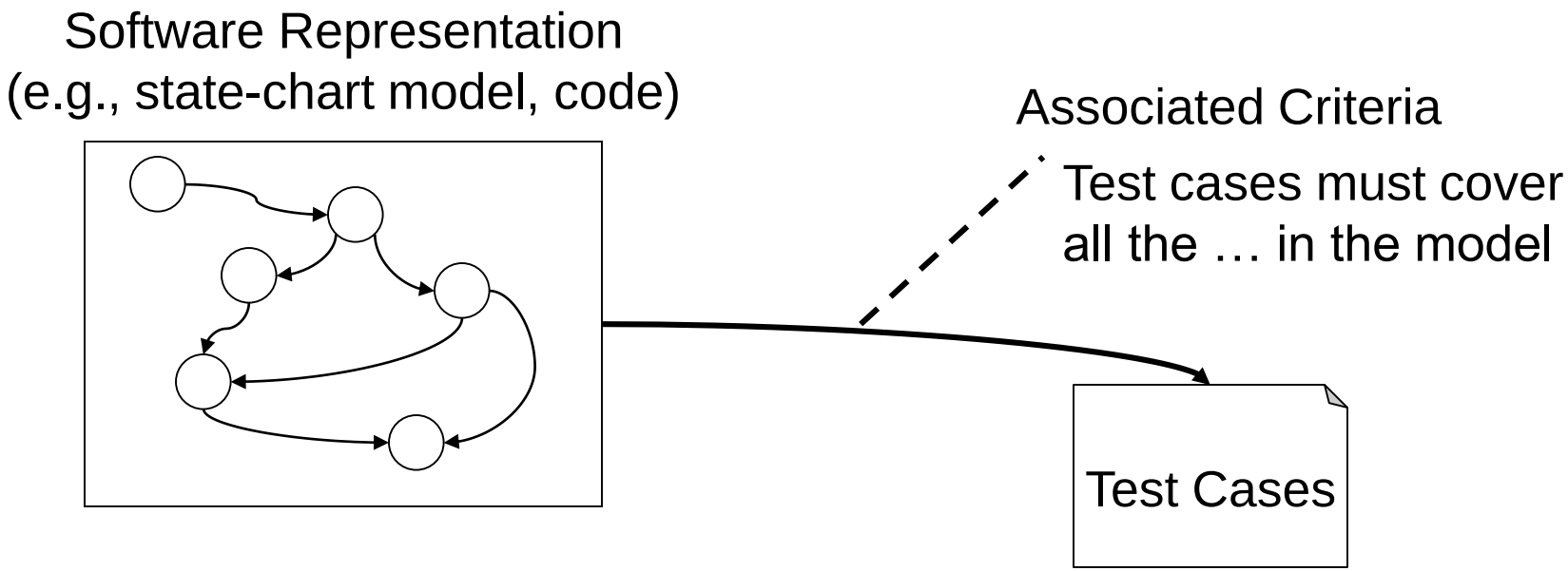
- All use cases
- All decision paths
- All menu items
- All states
- All boundary values ...

Structural (white-box)

- All methods
- All statements
- All code branches (if/else)
- …

A test-adequacy criterion leads to **test obligations**.
Test coverage: percentage of obligations that are met.

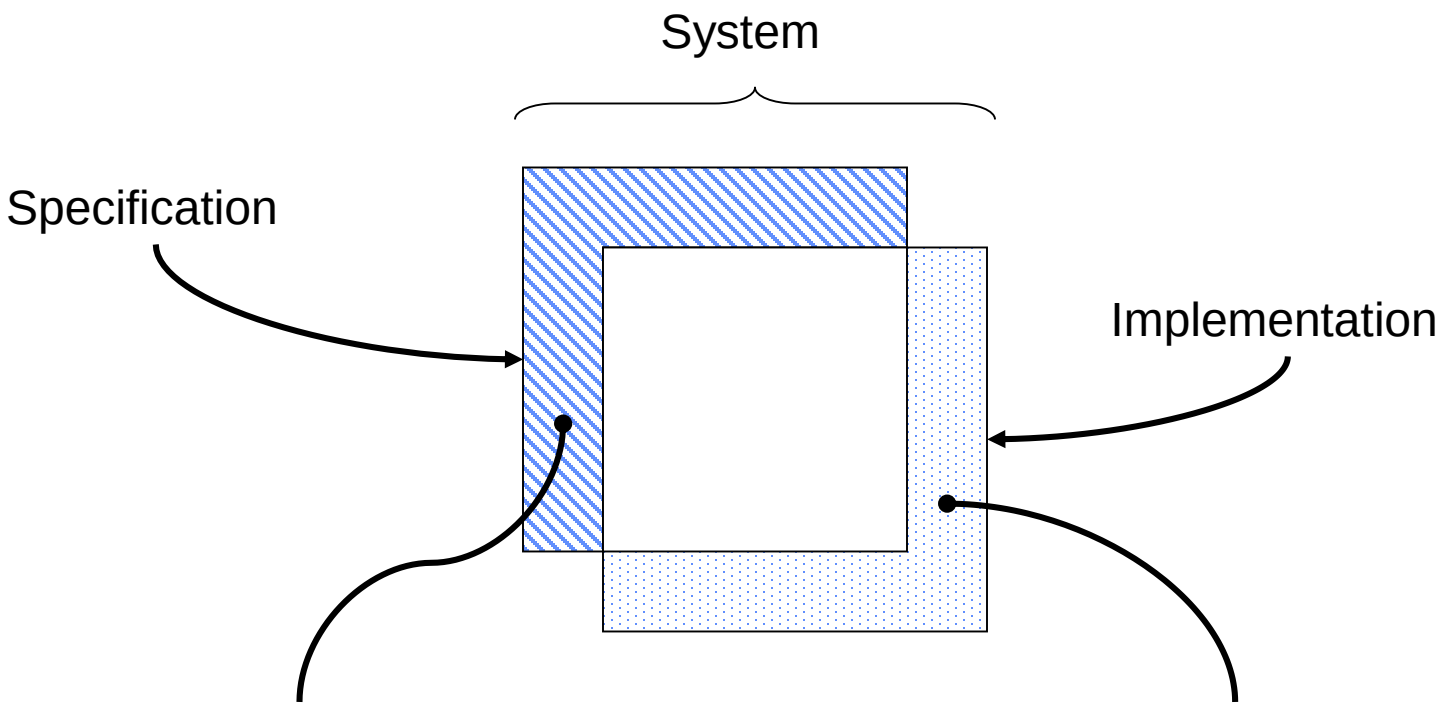
Test Coverage and Types of Testing



Types of testing: Whether test cases are generated based on:

- **<u>Specification</u> ⇒ Black-Box Testing**
- **<u>Implementation</u> ⇒ White (glass)-Box Testing**

Black- vs. White (glass)-Box Testing



Missing functionality:
Cannot be revealed by white-box techniques

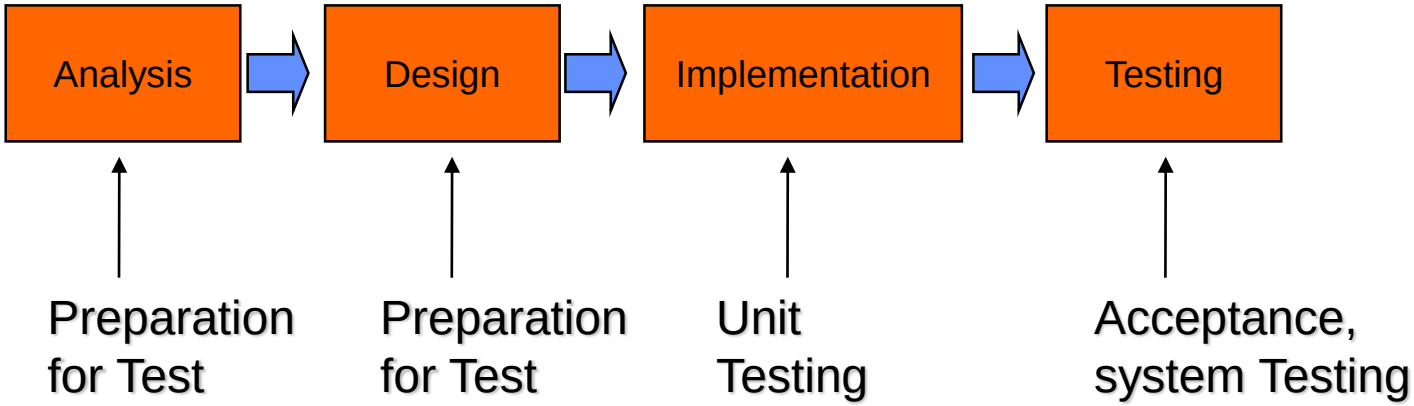
Unexpected functionality:
Cannot be revealed by black-box techniques

Introduction to Software Testing- Outline

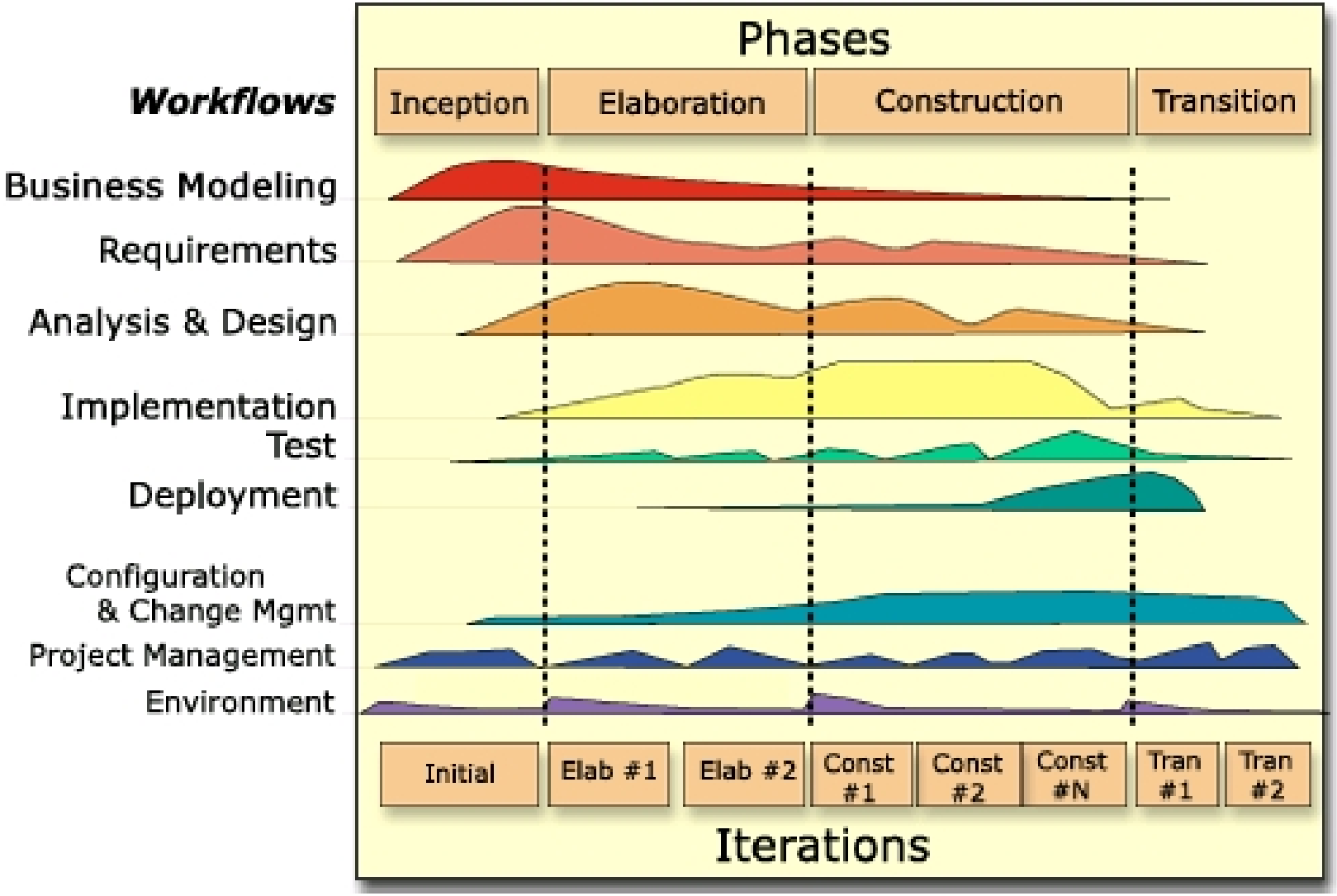
- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- **Testing throughout the Software Development Life Cycle**
- Testing Maturity Levels

Testing throughout the SDLC – in Waterfall

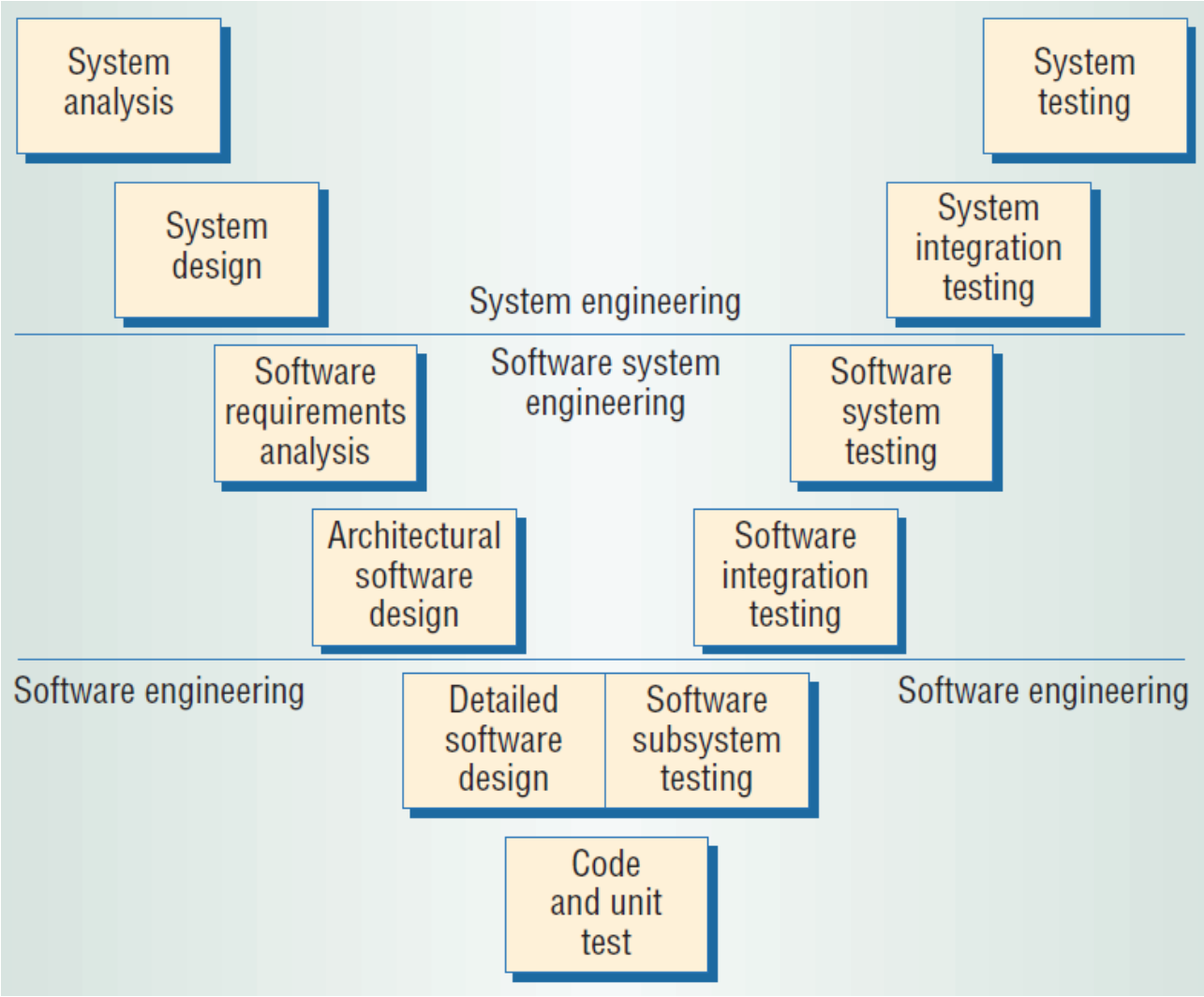
- SDLC: Software Development Life Cycle
- Much of the life-cycle development artifacts provides a rich source of test data
- Identifying test requirements and test cases early helps shorten the development time
- They may help reveal faults
- It may also help identify early low testable specifications or design



Testing throughout the Lifecycle – in Rational Unified Process (RUP)

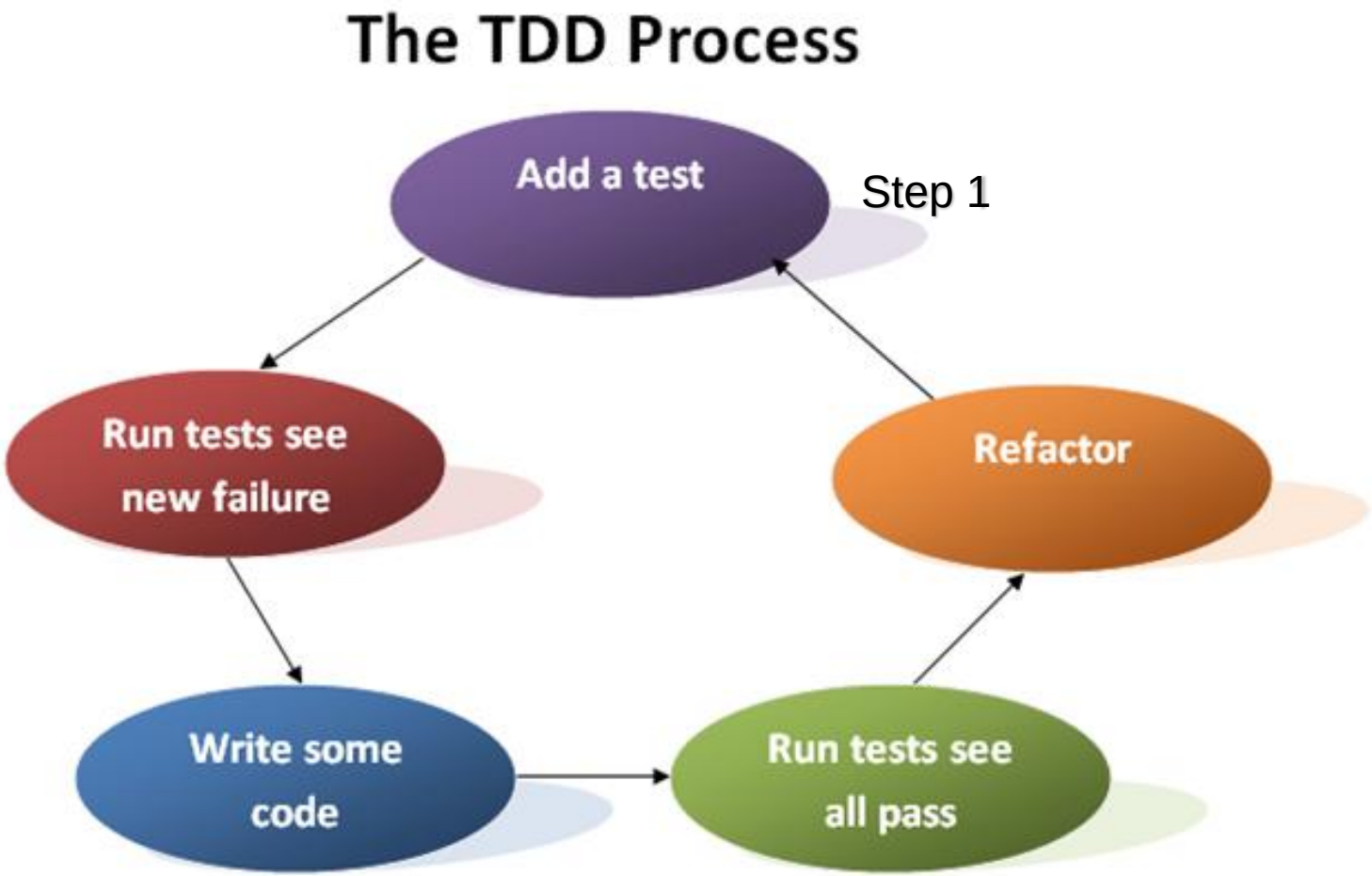


Testing throughout the Lifecycle – in the "V" model (half of this is about testing!)



Testing throughout the Lifecycle – in TDD

(Test-Driven Development)

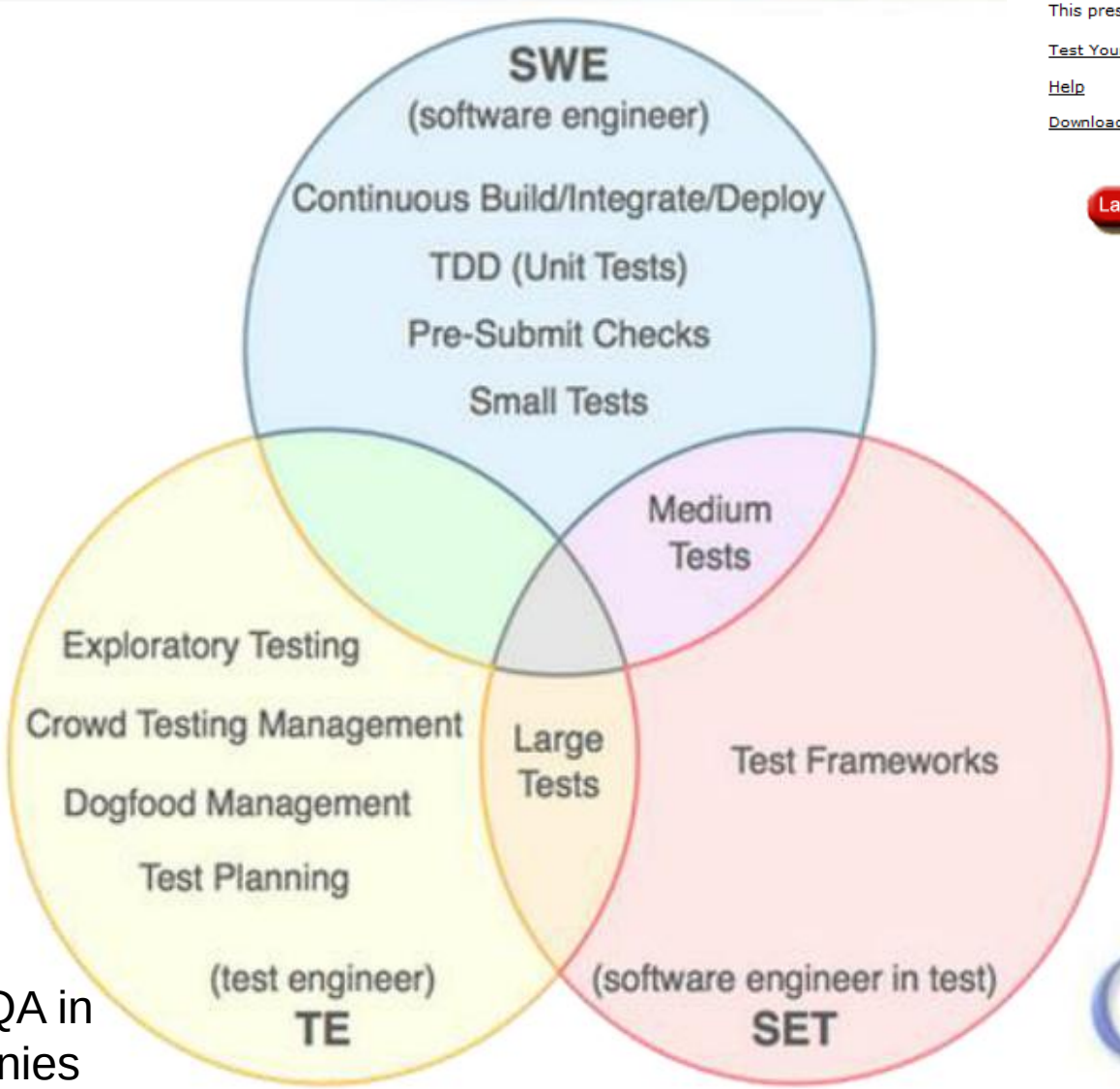


In TDD, test cases are actually the requirements of the system.

Introduction to Software Testing- Outline

- Definitions
- Why is SW testing important?
- Examples of Software Failures
- Dealing with SW Faults
- Testing Process Overview
- What is a Test Case and Test Suite (Test Set)?
- Types of Test Activities
- Types of Testing
- Test Levels/ Test Organization
- Testing can be Manual or Automatic
- What is a good test case?
- Exhaustive (Complete) Testing
- Test Adequacy and Coverage: When can we say testing is enough?
- Testing throughout the Software Development Life Cycle
- Testing Maturity Levels
- **Testing Roles**

Testing roles



Also, called QA in many companies

WEBSEMINARS
A TECHWELL™ PRODUCTION

How Google Tests Software

Thank you for registering for this event!

This presentation will be available to audience members until M

Test Your System

Help

Download Flash Player

Launch Presentation

Google

SE 550-Software Engineering

Black-box (Functional) Testing

Vahid Garousi

Basic Principles

- Based on the definition of a program's specification, as opposed to its structure (code, interface)
- The notion of complete coverage can also be applied to functional (Black-box) testing
- Rigorous specifications have another benefit, they help Black-box testing, e.g., categorize inputs, derive expected outputs
- In other words, they help test case generation and test oracles

Black-box Testing: Different Techniques

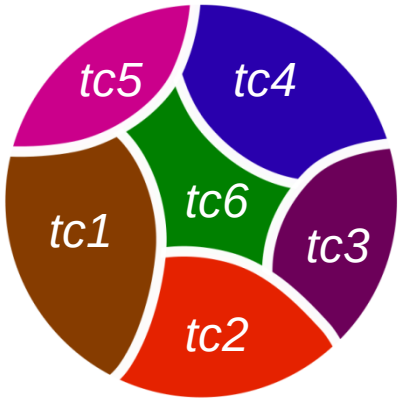
- **Equivalence Class Partitioning**
- **Boundary-Value Analysis**
- **Category-Partition**
- **Decision Tables**
- **Cause-Effect Graphs**

Black-box Testing: Outline

- **Equivalence Class Partitioning**
- **Boundary-Value Analysis**
- **Category-Partition**
- **Decision Tables**
- **Cause-Effect Graphs**

Equivalence Class Testing

- Motivation: we would like to have a sense of complete testing and we would hope to avoid test redundancy
- Equivalence classes: partitions of the input set in which input data have the same effect on the program (e.g., the result in the same output)
- Entire input set is covered: completeness



A SUT's input set

- Disjoint classes: to avoid redundancy
- Test cases: one element of each equivalence class
- But equivalence classes have to be chosen wisely …
- Guessing the likely system behavior might be needed

Companies are using Equivalence Class Testing!

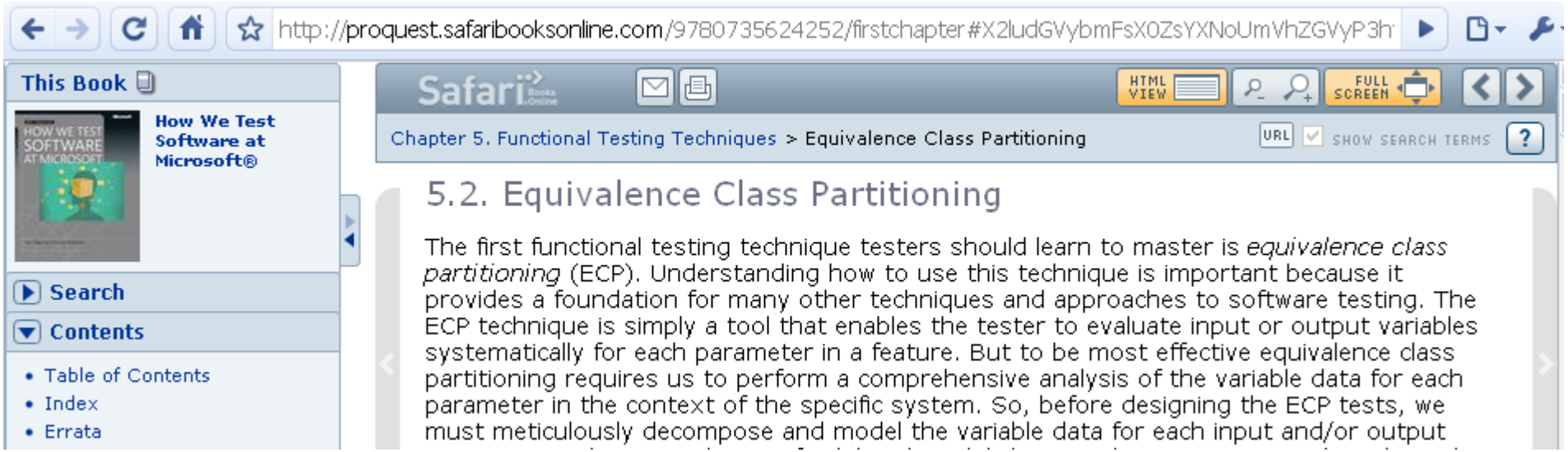


Figure 5-2. A C# implementation of the Next Date program.

Example of Equivalence Classes

- `NextDate` is a function with three variables: `month`, `day`, `year`. It returns the date of the day after the input date. Limitation: years 1812-2020
- Treatment Summary:
  - If it is not the last day of the month, the next date function will simply increment the day value.
  - At the end of a month, the next day is 1 and the month is incremented.
  - At the end of the year, both the day and the month are reset to 1, and the year incremented.
  - Finally, the problem of leap year makes determining the last day of a month interesting.
- Definition of leap years (http://en.wikipedia.org/wiki/Leap_years): The Gregorian calendar adds a 29th day to February in all years evenly divisible by 4, except for centennial years (those ending in -00) which are not evenly divisible by 400. Thus 1600, 2000 and 2400 are leap years but 1700, 1800, 1900, 2100, 2200 and 2300 are not.
- Only the year 1900 falls in the 1812-2020 period

`NextDate` Equivalence Classes

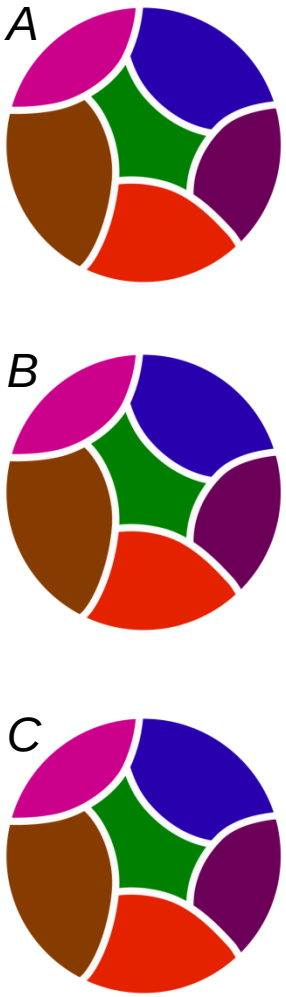
- M1 = {month | month has 30 days}
- M2 = {month | month has 31 days}
- M3 = {month | month is February (the impact of leap years)}

- D1 = {day | 1<= day <= 28}
- D2 = {day | day = 29}= 29
- D3 = {day | day = 30}=30
- D4 = {day | day = 31}=31

- Y1 = {year | year = 1900}= 1900
- Y2 = {year | 1812 <= year <= 2012 AND (year != 1900) AND (year mod 4 = 0)}
- Y3 = {year | (1812 <= year <= 2012 AND year mod 4 != 0)}

Weak/Strong Equivalence Class Testing

- For an example SUT, suppose there are three input variables from three domains: A, B, C
- $A = A_1 \cup A_2 \cup A_3 \cup \ldots \cup A_m$ where $a_i \in A_i$
- $B = B_1 \cup B_2 \cup B_3 \cup \ldots \cup B_n$ where $b_i \in B_i$
- $C = C_1 \cup C_2 \cup C_3 \cup \ldots \cup C_o$ where $c_i \in C_i$
- Weak Equivalence Class Testing: Choosing one variable value form each equivalence class (one a_i, b_i, and c_i) such that all classes are covered. # of test cases?
  - max (|A|, |B|, |C|)
- Strong Equivalence Class Testing: Is based on the Cartesian product of the partition subsets ($A \times B \times C$), i.e., testing all interactions of all equivalence classes. # of test cases?
  - |A| x |B| x |C|
- Examples next…



Example of Weak Equivalence Class Testing (WECT)

- *For an example SUT, suppose there are three input variables from three domains: A, B, C*
- $A = A_1 \cup A_2 \cup A_3$ *where* $a_i \in A_i$
- $B = B_1 \cup B_2 \cup B_3 \cup B_4$ *where* $b_i \in B_i$
- $C = C_1 \cup C_2$ *where* $c_i \in C_i$
- *Number of WETCs needed=Max size w.r.t. the number of equivalence classes of {A, B, C}*
- *4 WETCs are enough.*

Test Case	A	B	C
WETC1	a1	b1	c1
WETC2	a2	b2	c2
WETC3	a3	b3	c1
WETC4	a1	b4	c2

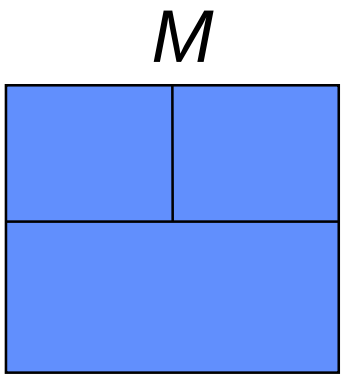
Example of Strong Equivalence Class Testing (SECT)

- *|A| (number of equivalence classes) = 3*
- *|B| = 4*
- *|C| = 2*

- *# of test cases=3x4x2=24*

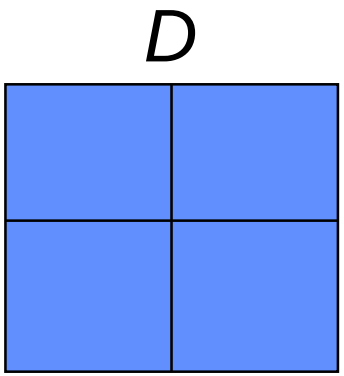
Test Case	A	B	C
SETC1	a1	b1	c1
SETC2	a1	b1	c2
SETC3	a1	b2	c1
SETC4	a1	b2	c2
SETC5	a1	b3	c1
SETC6	a1	b3	c2
SETC7	a1	b4	c1
SETC8	a1	b4	c2
SETC9	a2	b1	c1
SETC10	a2	b1	c2
SETC11	a2	b2	c1
SETC12	a2	b2	c2
SETC13	a2	b3	c1
SETC14	a2	b3	c2
SETC15	a2	b4	c1
SETC16	a2	b4	c2
SETC17	a3	b1	c1
SETC18	a3	b1	c2
SETC19	a3	b2	c1
SETC20	a3	b2	c2
SETC21	a3	b3	c1
SETC22	a3	b3	c2
SETC23	a3	b4	c1
SETC24	a3	b4	c2

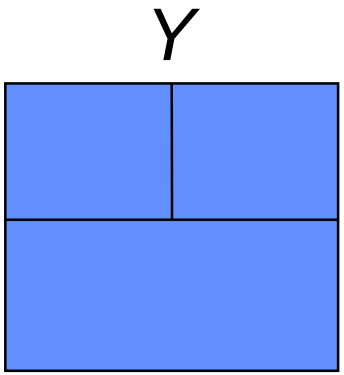
`NextDate` Equivalence Classes



- M1 = {month | month has 30 days}
- M2 = {month | month has 31 days}
- M3 = {month | month is February (the impact of leap years)}

- D1 = {day | 1<= day <= 28}
- D2 = {day | day = 29}
- D3 = {day | day = 30}
- D4 = {day | day = 31}





- Y1 = {year | year = 1900}
- Y2 = {year | 1812 <= year <= 2012 AND (year != 1900) AND (year mod 4 = 0)}
- Y3 = {year | (1812 <= year <= 2012 AND year mod 4 != 0)}

Weak Equivalence Class Testing (WECT) - Test Cases

- Number of WECT test cases=maximum partition size (D)=4

Test Case ID	Month	Day	Year	Output
WETC1	6	14	1900	6/15/1900
WETC2	7	29	1912	7/30/1912
WETC3	2	30	1913	Invalid Input date (not possible)
WETC4	5	31	1900	Invalid Input date

Strong Equivalence Class Testing (SECT) - Test Cases

- *Number of SECT test cases= partition size (D) x partition size (M) x partition size (Y) = 3x4x3=36 test cases*

Test Case ID	Month	Day	Year	Expected Output
SE1	6	14	1900	6/15/1900
SE2	6	14	1912	6/15/1912
SE3	6	14	1913	6/15/1913
SE4	6	29	1900	6/30/1900
SE5	6	29	1912	6/30/1912
SE6	6	29	1913	6/30/1913
SE7	6	30	1900	7/1/1900
SE8	6	30	1912	7/1/1912
SE9	6	30	1913	7/1/1913
SE10	6	31	1900	ERROR
SE11	6	31	1912	ERROR
SE12	6	31	1913	ERROR
SE13	7	14	1900	7/15/1900
SE14	7	14	1912	7/15/1912
SE15	7	14	1913	7/15/1913
SE16	7	29	1900	7/30/1900
SE17	7	29	1912	7/30/1912
SE18	7	29	1913	7/30/1913
SE19	7	30	1900	7/31/1900
SE20	7	30	1912	7/31/1912
SE21	7	30	1913	7/31/1913
SE22	7	31	1900	8/1/1900
SE23	7	31	1912	8/1/1912
SE24	7	31	1913	8/1/1913
SE25	2	14	1900	2/15/1900
SE26	2	14	1912	2/15/1912
SE27	2	14	1913	2/15/1913
SE28	2	29	1900	ERROR
SE29	2	29	1912	3/1/1912
SE30	2	29	1913	ERROR
SE31	2	30	1900	ERROR
SE32	2	30	1912	ERROR
SE33	2	30	1913	ERROR
SE34	2	31	1900	ERROR
SE35	2	31	1912	ERROR
SE36	2	31	1913	ERROR

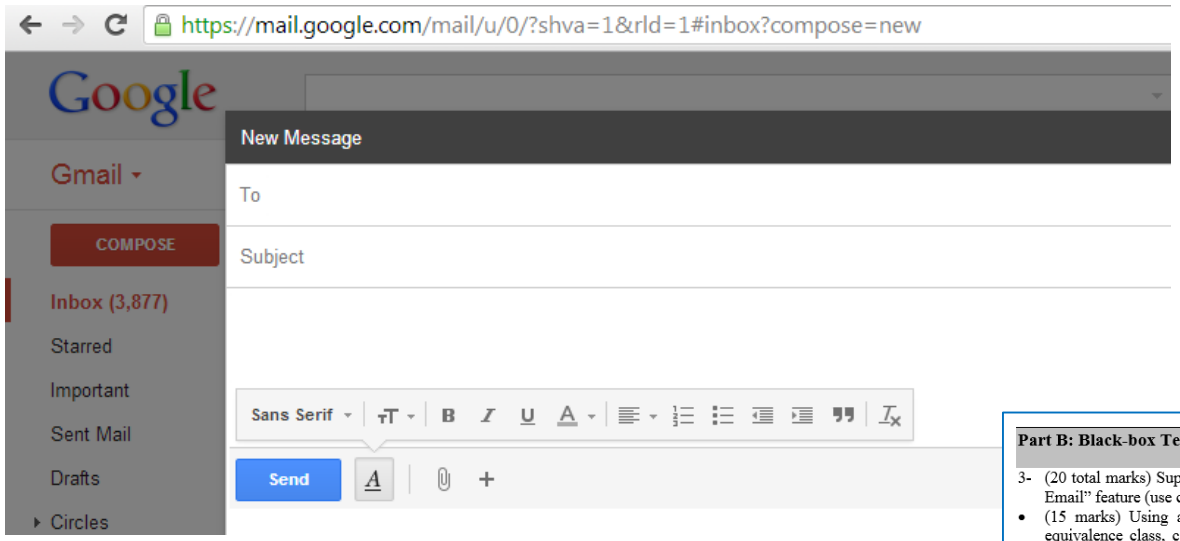
Equivalence Class Testing

`NextDate` Example - Discussion

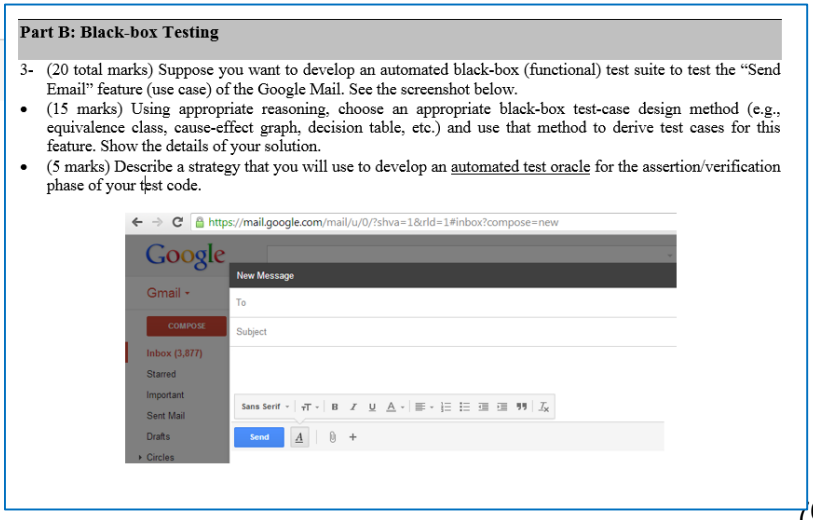
- If error conditions are a high priority, we should extend strong equivalence class testing to include invalid classes
  - **E.g., year | year < 1812 or year > 2012**
- Equivalence Class Testing is appropriate when input data defined in terms of ranges and sets of discrete values
- We can also consider extra equivalence classes to consider the “invalid” situations, e.g., years before 1812 or after 2012

Exercise

- Use equivalence-class testing to develop an automated black-box (functional) test suite to test the "*Send Email*" feature (use case) of the Google Mail. See the screenshot below.



- Was in exam a few years ago!



Part B: Black-box Testing

3- (20 total marks) Suppose you want to develop an automated black-box (functional) test suite to test the "Send Email" feature (use case) of the Google Mail. See the screenshot below.

- (15 marks) Using appropriate reasoning, choose an appropriate black-box test-case design method (e.g., equivalence class, cause-effect graph, decision table, etc.) and use that method to derive test cases for this feature. Show the details of your solution.
- (5 marks) Describe a strategy that you will use to develop an automated test oracle for the assertion/verification phase of your test code.

FOR STUDENT READING

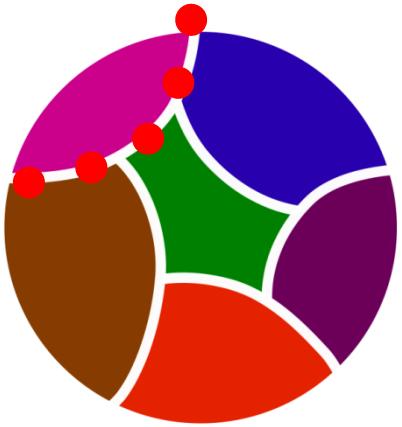
Black-box Testing: Outline

- Equivalence Class Partitioning
- **Boundary-Value Analysis**
- Category-Partition
- Decision Tables
- Cause-Effect Graphs

Boundary-Value Analysis

Motivations

- In equivalence class testing, we partitioned input domains into equivalence classes, on the assumption that the behavior of the program is “similar” for all input values of a equivalence class
- But the above assumption may not be true in all cases as…
- Some typical programming errors happen to be at the <u>boundary</u> between different equivalence classes
- This is what boundary value testing focuses on
- Simpler but complementary to equivalence class testing



http://proquest.safaribooksonline.com/9780735624252/firstchapter#X2ludGVybmFsX0ZsYXNoUmVhZGVyP3h

Safari Books Online

HTML VIEW | FULL SCREEN

Chapter 5. Functional Testing Techniques > Boundary Value Analysis

URL | SHOW SEARCH TERMS | ?

This Book

How We Test Software at Microsoft®

Search

Contents

- Table of Contents
- Index
- Errata

Copyright
Acknowledgments
Introduction
Pt. I. About Microsoft
Pt. II. About Testing
Ch. 4. A Practical Approach to Test Case Design
Ch. 5. Functional Testing Techniques

5.3. Boundary Value Analysis

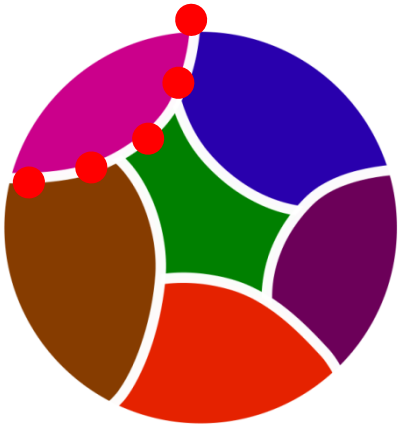
Boundary value analysis (BVA) is perhaps the best-known functional test technique. Unfortunately, it is also typically misused because many testers assume it to be relatively easy or trivial. Historical evidence demonstrates that a surprising number of problems occur at boundaries of linear variables, so we must carefully analyze the boundary conditions of linear variable data to avoid overlooking boundary class defects. When BVA is used in conjunction with equivalence class partitioning, the BVA functional technique can be very effective in systematically analyzing the boundary values of linear variables for independent input and output parameters. BVA or boundary testing is especially useful for detecting the following types of errors:

- Incorrect artificial constraints of a data type
- Erroneously assigned relational operators
- Wrapping of data types
- Problems with looping structures
- Off-by-one errors

Boundary-Value Analysis

Basic Ideas

- Assume a function F, with two variables x1 and x2
- (Possibly unstated) boundaries: a <= x1 <= b, c <= x2 <= d
- In some programming languages, strong typing allows the specification of such intervals
- In boundary-value analysis (and testing), the focus is on the boundary of the input space for identifying test cases
- The rationale is that errors tend to occur near extreme values of input variables



Boundary-Value Analysis

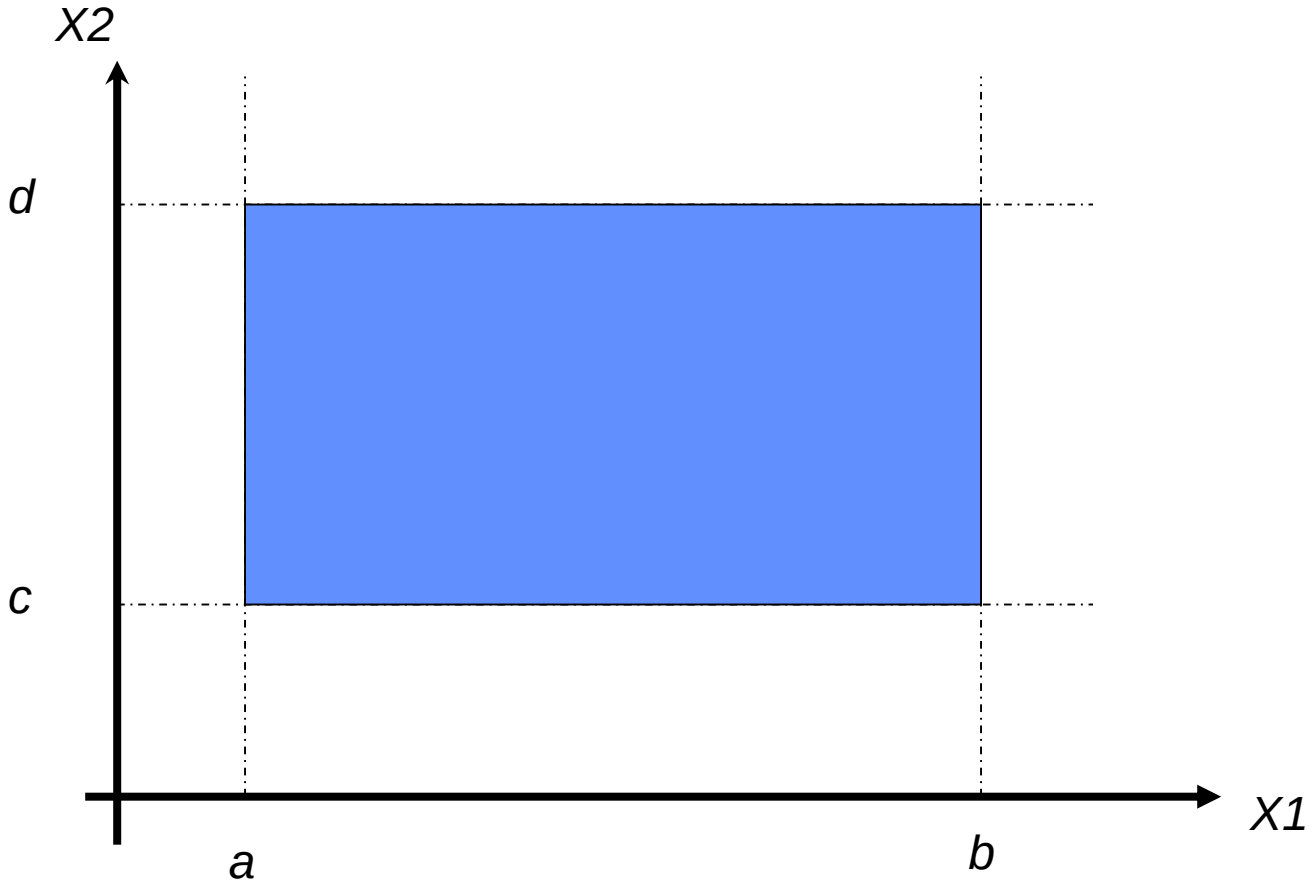
Basic Ideas

- Approach: Setting values for input variable at their minimum, just above the minimum, a nominal value, just below their maximum, and at their maximum.
- Convention for the above notions:
  - min, min+, nom, max-, max
- A usual strategy for all input variables: Holding the values of all but one variable at their nominal values, letting one variable assume its extreme value
- Examples next…

Boundary-Value Analysis

Input Domain of Function F

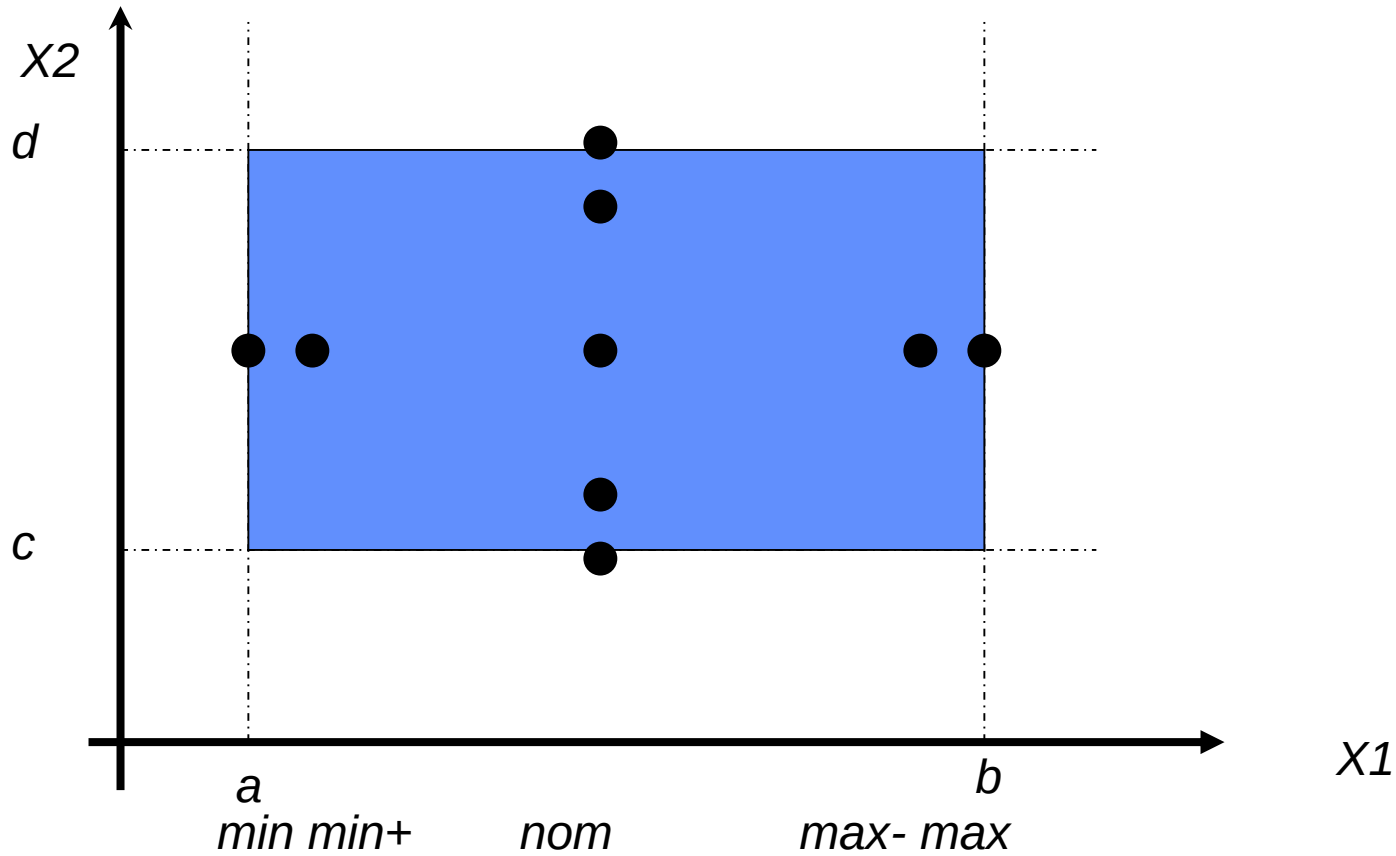
- *Assume a function F, with two variables x1 and x2, domains X1 and X2*
- *(Possibly unstated) boundaries: a <= x1 <= b, c <= x2 <= d*



Boundary-value Analysis

Test Cases

- Test set ={ <$x1_{nom}$, $x2_{min}$>, <$x1_{nom}$, $x2_{min+}$>, <$x1_{nom}$, $x2_{nom}$>, <$x1_{nom}$, $x2_{max-}$>, <$x1_{nom}$, $x2_{max}$>, <$x1_{min}$, $x2_{nom}$,>, <$x1_{min+}$, $x2_{nom}$,>, <$x1_{max-}$, $x2_{nom}$>, <$x1_{max}$, $x2_{nom}$>}.
- Number of test cases=9



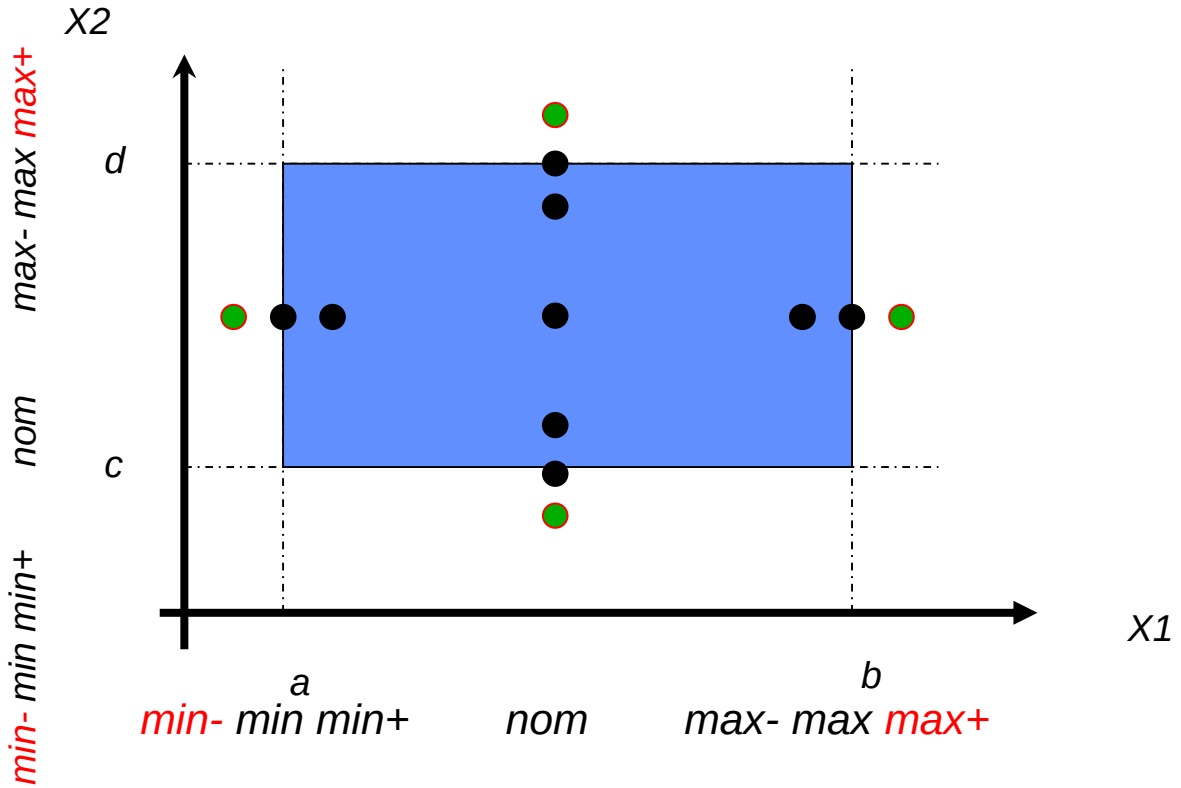
Boundary-Value Analysis

General Case and Limitations

- A function with n variables will require 4n + 1 test cases
- Works well with variables that represent bounded physical quantities
- No consideration of the nature of the function and the meaning of variables
- An elementary technique that is a extendable to robustness testing (next)
  - "Robustness is defined as the degree to which a system operates correctly in the presence of *exceptional inputs* or *stressful environmental conditions*." [IEEE Standard 610.12.1990]
  - The goal of robustness testing is to develop test cases and test environments where a system's robustness can be assessed.

Boundary-Value Analysis: The relationship with Robustness Testing

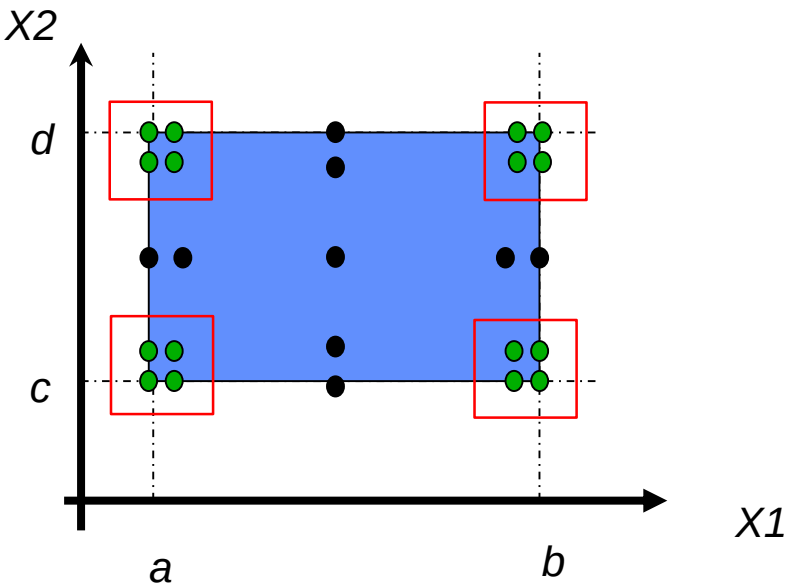
- In Robustness Testing, We also look at the behavior of the system when the variable extremes are exceeded with a value slightly greater than the maximum (max+) and a value slightly less than the minimum (min-)
- This is an extension of the basic boundary-value testing.
- A function with n variables will require 6n + 1 test cases



Boundary-Value Analysis

Worst Case Testing (WCT)

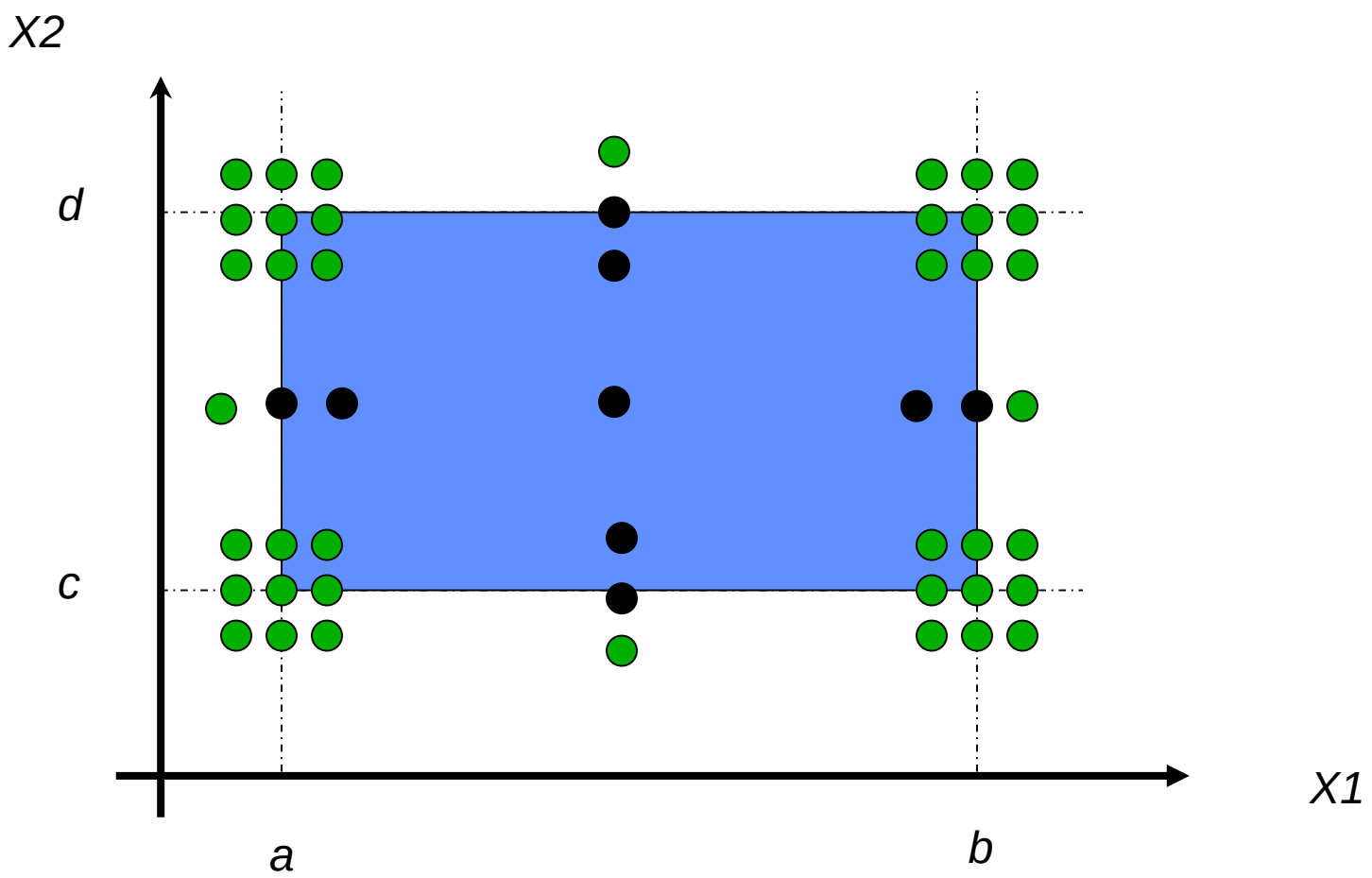
- Boundary-value makes the common assumption that failures, most of the time, originate from one fault
- What happens when more than one variable has an extreme value?
- The idea comes from electronics in circuit analysis
- Cartesian product of {min, min+, nom, max-, max}
- *Clearly more thorough than boundary-value analysis, but with much more effort: 5^n test cases (n: number of variables)*
- *WCT is a good strategy when physical variables have numerous interactions, and where failure is costly*



Boundary-Value Analysis

Even further: Robust Worst-Case Testing for 2 variables

- 7^n test cases (n: number of variables)



Black-box Testing: Outline

- Equivalence Class Partitioning
- Boundary-Value Analysis
- **Category-Partition Testing**
- Decision Tables
- Cause-Effect Graphs

Category-Partition Testing

Steps

- The system is divided into individual “functions” that can be independently tested
- The method identifies the *parameters* of each “function” and, for each parameter, identifies distinct *categories*
- Besides parameters, *environment characteristics,* under which the function operates (characteristics of the system state), can also be considered
- *Categories* are major properties or characteristics for each parameter
- The *categories* are further subdivided into *choices* in the same way as equivalence partitioning is applied (possible “values”)
- A broader idea compared to Equivalence-Class Partitioning. (see the notes area)
- In essence: Category-partition testing integrates equivalence class testing, boundary analysis, and adds a few refinements

Category-Partition Testing

An Example

- Specification: The program prompts the user for a positive integer in the range 1 to 20 and then for a string of characters of that length.
- The program then prompts for a character and returns either the position in the string at which the character was first found or a message indicating that the character was not present in the string.
- The user has the option to search for more characters.

Category-Partition Testing Example

Parameters and Categories

- Parameters: Three
  - integer x (length), the string a, and the character c
- Categories:
  - For the length x, the categories are “in-range” (1-20) or “out-of-range”
  - Categories for the string a: minimal, maximal, intermediate length
  - Categories for character c: character appears at the beginning, middle, end of string, or does not occur in the string

Category-Partition Testing Example

Choices

- For Integer x, out-of-range: 0, 21
- For Integer x, in-range: 1, 2-19, 20
- For String a: 1, 2-19, 20
- For Character c: first, middle, last, does not occur

- Note: sometimes we can have only one choice in each category

Category-Partition Testing Example

Formal Test Specifications with Constraints

Recall the definition of dependent and independent variables from statistics

x (length):

Choices

1)	0	[error (out of range)]
2)	1	[property stringOk, MinLength]
3)	2-19	[property stringOk, MidLength]
4)	20	[property stringOk, MaxLength]
5)	21	[error (out of range)]

independent

a (string):

Choices

1)	Of length 1	[if stringOk and MinLength]
2)	Of length 2-19	[if stringOk and MidLength]
3)	Of length 20	[if stringOk and MaxLength]

dependent

c (character):

Choices

1)	At first position in string	[if stringOk]
2)	At last position in string	[if stringOk and not MinLength]
3)	In middle of string	[if stringOk and not MinLength]
4)	Not in string	[if stringOk]

Category-Partition Testing

Specifying constraints among choices, in General

- *Properties,* and *Selectors* are associated with choices

3 properties (definitions)

Category A

Choices

ChoiceA1 [property X, Y, Z]

ChoiceA2

Category B

A selector

ChoiceB1

ChoiceB2 [if X and Z]

- E.g., in the above abstract example, ChoiceA2 and ChoiceB2 would not be combined into a test frame

Category-Partition Testing Example

Test Frames, Data and Cases

- The number depends on the inter-choice constraint dependencies
- Without constraints: 5*3*4 = 60 test frames/cases (i.e., using SECT)
- We have reduced to 12 using constraints

x (length):

1)	*0*	*[error (out of range)]*
2)	*1*	*[property stringOk, MinLength]*
3)	*2-19*	*[property stringOk, MidLength]*
4)	*20*	*[property stringOk, MaxLength]*
5)	*21*	*[error (out of range)]*

a (string):

1)	*Of length 1*	*[if stringOk and MinLength]*
2)	*Of length 2-19*	*[if stringOk and MidLength]*
3)	*Of length 20*	*[if stringOk and MaxLength]*

c (character):

1)	*At first position in string*	*[if stringOk]*
2)	*At last position in string*	*[if stringOk and not MinLength]*
3)	*In middle of string*	*[if stringOk and not MinLength]*
4)	*Not in string*	*[if stringOk]*

x1	***x = 0***
x2a1c1	***x = 1, a = 'A', c = 'A'***
x2a1c4	***x = 1, a = 'A', c = 'B'***
x3a2c1	***x = 7, a = 'ABCDEFG', c = 'A'***
x3a2c2	***x = 7, a = 'ABCDEFG', c = 'G'***
x3a2c3	***x = 7, a = 'ABCDEFG', c = 'D'***
x3a2c4	***x = 7, a = 'ABCDEFG', c = 'X'***
x4a3c1	***x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'A'***
x4a3c2	***x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'T'***
x4a3c3	***x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'J'***
x4a3c4	***x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'X'***
x5	***x = 21***

Category-Partition Testing

Steps (II)

- The *constraints* operating between choices are then identified, i.e., how the occurrence of one choice can affect the existence of another
  - E.g., in the array sorting example, if Len = 0, then the rest does not matter
- *Test frames* (or test specifications) are then generated which are defined as the allowable combinations of choices in the categories
- Test frames are then converted into *test data*

Criteria Using Choices

- All Combinations (AC): This is what was shown in the previous example, what is typically done when using category-partition. One value for every choice of every parameter must be used with one value of every (possible) choice of every other category.
- Each choice (EC): This is a weaker criterion. One value from each choice for each category must be used at least in one test case.
  - **Should use a "greedy approach"**
  - **How many test cases for our example? 7 TCs**

x1	x = 0
x2a1c1	x = 1, a = 'A', c = 'A'
~~x2a1c4~~	~~x = 1, a = 'A', c = 'B'~~
x3a2c1	x = 7, a = 'ABCDEFG', c = 'A'
~~x3a2c2~~	~~x = 7, a = 'ABCDEFG', c = 'G'~~
~~x3a2c3~~	~~x = 7, a = 'ABCDEFG', c = 'D'~~
~~x3a2c4~~	~~x = 7, a = 'ABCDEFG', c = 'X'~~
~~x4a3c1~~	~~x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'A'~~
x4a3c2	x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'T'
x4a3c3	x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'J'
x4a3c4	x = 20, a = 'ABCDEFGHIJKLMNOPQRST', c = 'X'
x5	x = 21

Conclusions

- Identifying parameters and environments conditions, and categories, heavily relies on the experience of the tester
- Makes testing decisions explicit (e.g., constraints), open for review
- Combines boundary-value analysis, robustness testing, and equivalence-class partitioning
- Once the first step (i.e., specifying the constraints) is completed, the technique is straightforward and can be automated
- The criteria for test-case reduction makes it useful for practical testing
- Source:

P. Ammann and J. Offutt, "Using Formal Methods to Derive Test Frames in Category-Partition Testing," *Safety, Reliability, Fault Tolerance, Concurrency, and Real Time Security: Proc. 9th Ann. Conf. Computer Assurance* (*COMPASS '94*), pp. 69–79. Los Alamitos, Calif.: IEEE Computer Soc. Press, 1994.

Black-box Testing: Outline

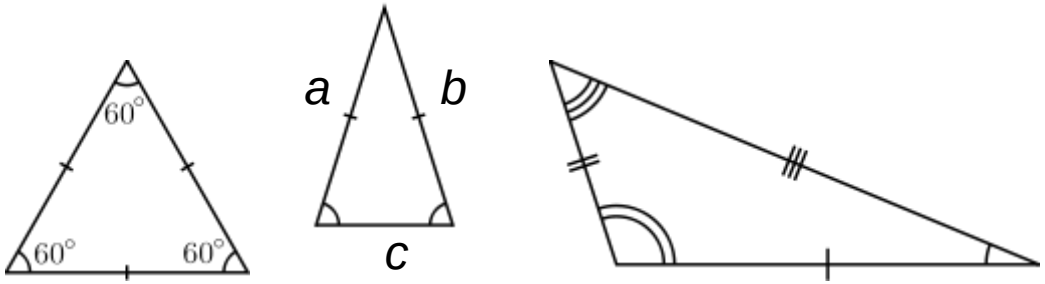
- Equivalence Class Partitioning
- Boundary-Value Analysis
- Category-Partition Testing
- **Decision Tables**
- Cause-Effect Graphs

Decision Tables - Motivations

- To help express *test requirements* in a directly usable form
- Easy to understand and support the systematic derivation of test cases
- To support automated or manual generation of test cases
- A particular response or response subset is to be selected for testing by evaluating many related conditions
- Ideal for describing situations in which a number of combinations of actions are taken under varying sets of conditions, e.g., control systems

Decision Tables - An Example

For a program identifying the type of a triangle given its edge (side) sizes



Equilateral Isosceles Scalene

Conditions

Decisions

Actions

c1: a, b, c triangle?	N					Y			
c2: a = b?	–			Y				N	
c3: a = c?	–		Y		N		Y		N
c4: b = c?	–	Y	N	Y	N	Y	N	Y	N
a1: not a triangle?	X								
a2: Scalene									X
a3: Isosceles					X		X	X	
a4: Equilateral		X							
a5: Impossible			X	X		X			

Decision Tables - Structure

c1: a, b, c triangle?	N					Y			
c2: a = b?	–			Y				N	
c3: a = c?	–		Y		N		Y		N
c4: b = c?	–	Y	N	Y	N	Y	N	Y	N
a1: not a triangle?	X								
a2: Scalene									X
a3: Isosceles					X		X	X	
a4: Equilateral		X							
a5: Impossible			X	X		X			

- There is a condition section which lists *conditions* and combinations thereof
- Conditions express relationships among *decision variables*
- There is an Action section which lists *responses* to be produced when corresponding combinations of conditions are true
- A limitation: Resultant actions are determined by the *current* values of decision variables!
- Actions are *independent* of input order and the order in which conditions are evaluated
- Actions may appear more than once but each combination of conditions is unique

Decision Tables - Table Structure

Conditions	c1			True		Fals	
	c2	True		False	True		False
	c3	T	F	–	T	F	–
Actions	a1	X	X		X		
	a2	X				X	
	a3		X		X	X	
	a4			X			X

Rule

Decision Tables - An Example

Can also be shown as a conventional Truth Table

c1: a, b, c triangle?	N					Y				
c2: a = b?	–			Y					N	
c3: a = c?	–		Y		N			Y		N
c4: b = c?	–	Y	N	Y	N	Y	N	Y	N	N
a1: not a triangle?	X									
a2: Scalene										X
a3: Isosceles					X		X	X		
a4: Equilateral		X								
a5: Impossible			X	X		X				

conditions											
c1: a < b + c?	F	T	T	T	T	T	T	T	T	T	T
c2: b < a + c?	-	F	T	T	T	T	T	T	T	T	T
c3: c < a + b?	-	-	F	T	T	T	T	T	T	T	T
c4: a = b?	-	-	-	T	T	T	T	F	F	F	F
c5: a = c?	-	-	-	T	T	F	F	T	T	F	F
c6: b = c?	-	-	-	T	F	T	F	T	F	T	F
a1: Not a triangle	X	X	X								
a2: Scalene											X
a3: Isosceles							X		X	X	
a4: Equilateral				X							
a5: Impossible					X	X		X			

Decision Tables - An Example

Test Cases

conditions											
c1: a < b + c?	F	T	T	T	T	T	T	T	T	T	T
c2: b < a + c?	-	F	T	T	T	T	T	T	T	T	T
c3: c < a + b?	-	-	F	T	T	T	T	T	T	T	T
c4: a = b?	-	-	-	T	T	T	T	F	F	F	F
c5: a = c?	-	-	-	T	T	F	F	T	T	F	F
c6: b = c?	-	-	-	T	F	T	F	T	F	T	F
a1: Not a triangle	X	X	X								
a2: Scalene											X
a3: Isosceles							X		X	X	
a4: Equilateral				X							
a5: Impossible					X	X		X			

A test rule

11 test rules…

One test case for each test rule from the corresponding truth table.

Test Case ID	a	b	c	Expected Output
TC1	4	1	2	Not a triangle
TC 2	1	4	2	Not a triangle
TC 3	1	2	4	Not a triangle
TC 4	5	5	5	Equilateral
TC 5	?	?	?	Impossible
TC 6	?	?	?	Impossible
TC 7	2	2	3	Isosceles
TC 8	?	?	?	Impossible
TC 9	2	3	2	Isosceles
TC 10	3	2	2	Isosceles
TC 11	3	4	5	Scalene

Decision Tables - Ideal Usage Conditions

- One of several distinct responses (i.e., actions) is to be selected according to distinct cases of input variables
- These cases can be modeled by mutually-exclusive Boolean expressions on the input variables
- The response to be produced does not depend on the order in which input variables are set or evaluated (e.g., events are received)
- The response does not depend on prior inputs or outputs

Decision Tables - Explosion of Condition Variants

- For n conditions, there may be at most 2^n *variants* (unique combinations of conditions and actions)
- Our triangle Example…

conditions											
c1: a < b + c?	F	T	T	T	T	T	T	T	T	T	T
c2: b < a + c?	-	F	T	T	T	T	T	T	T	T	T
c3: c < a + b?	-	-	F	T	T	T	T	T	T	T	T
c4: a = b?	-	-	-	T	T	T	T	F	F	F	F
c5: a = c?	-	-	-	T	T	F	F	T	T	F	F
c6: b = c?	-	-	-	T	F	T	F	T	F	T	F

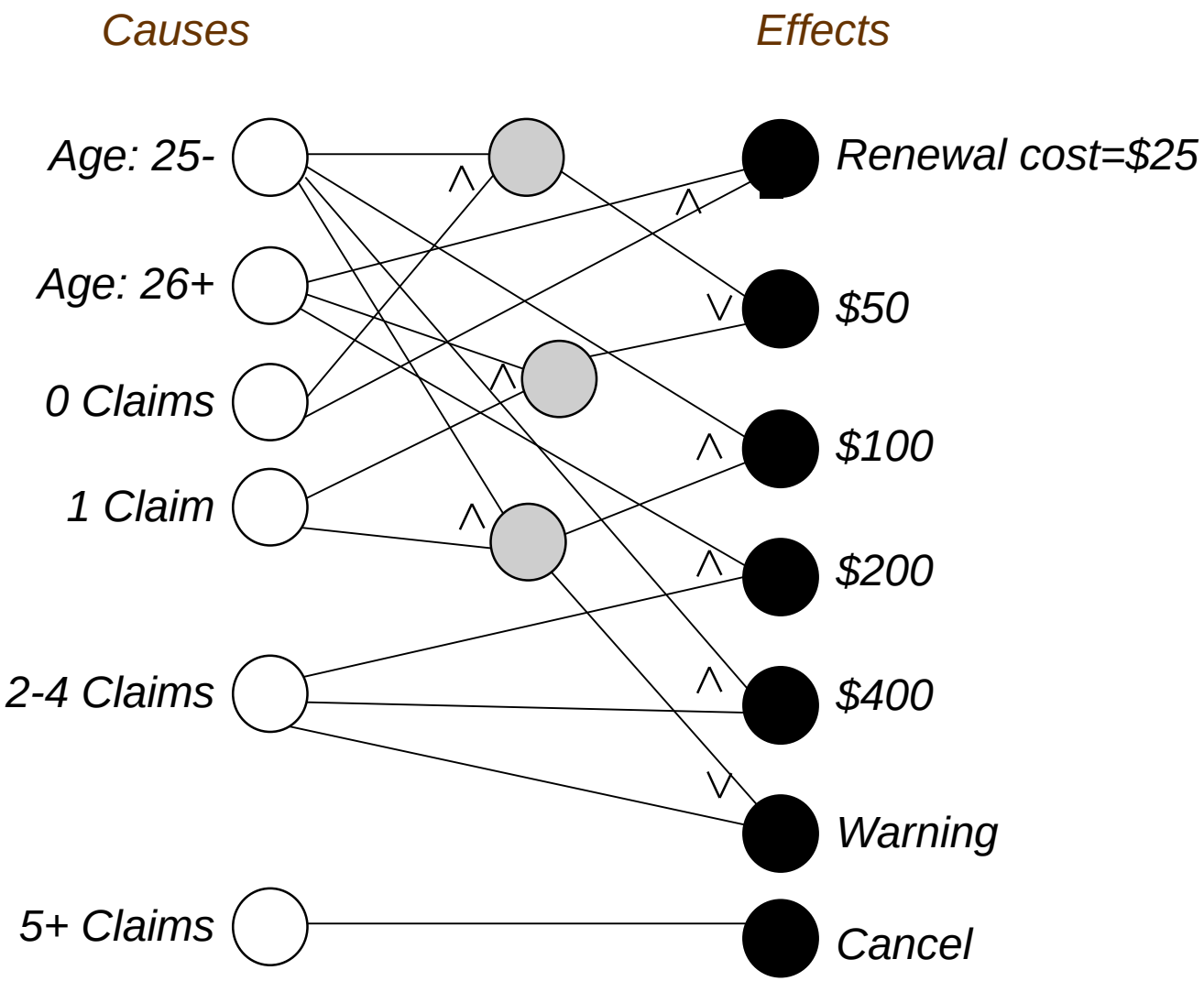
- But, fortunately, there are usually much fewer *explicit* variants in SW systems…
- "Don't care" values in decision tables help reduce the number of those variants
- "Don't care" can correspond to several cases:
  - The inputs are necessary but have no effect
  - The inputs may be omitted

Black-box Testing: Outline

- **Equivalence Class Partitioning**
- **Pair-wise and n-way testing**
- **Boundary-Value Analysis**
- **Category-Partition Testing**
- **Decision Tables**
- **Cause-Effect Graphs**

Cause-Effect Graphs

Car Insurance Renewal Example



Cause-Effect Graphs

Definition

- Graphical technique that helps derive decision tables
- Aim at supporting interaction with domain experts and the reverse engineering of specifications, for the purpose of testing.
- Identify causes (conditions on inputs, stimuli) and effects (outputs, changes in system state)
- Causes have to be stated in such a way to be either true or false (Boolean expression)
- Can be used to explicitly specify constraints (environmental, external) on causes and effects
- They can help select more “significant” subset of input-output combinations and build smaller decision tables

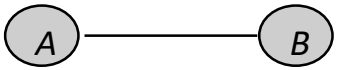
Cause-Effect Graphs

Structure of

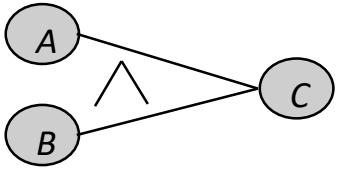
- A node is drawn for each cause and effect
- Nodes placed on opposite side of a sheet
- An edge from a cause to an effect indicates that the cause is a necessary condition for the effect
- If a single effect has two or more causes, the logical relationship of the causes is annotated by symbols for logical *and* (^) and logical *or* ($\vee$) placed between lines
- A cause whose negation is necessary is shown by a logical *not* (~)
- A single cause may be necessary for many effects; a single effect may have many necessary causes.
- Intermediate nodes may be used to simplify the graph and its construction

Cause-Effect Graphs

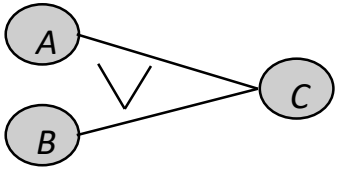
Notations in



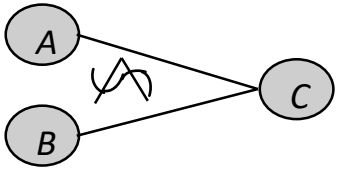
If A then B



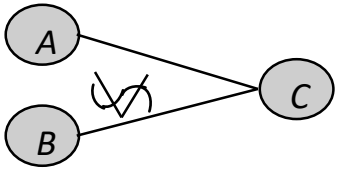
AND: If (A and B) then C



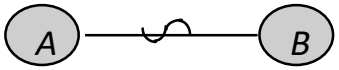
OR: If (A or B) then C



NAND: If not (A and B) then C



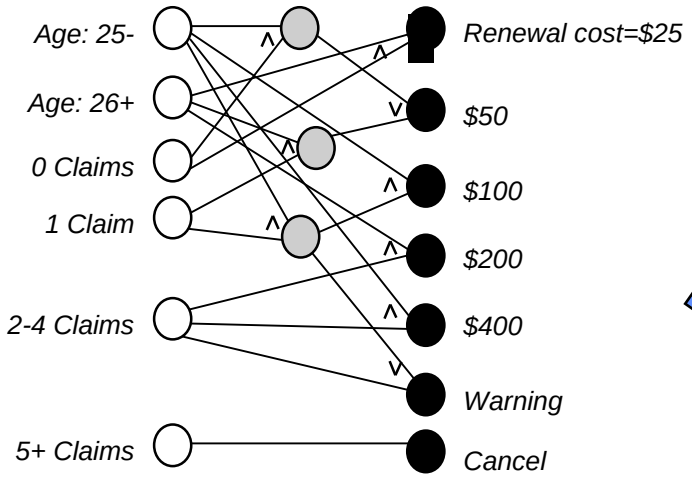
NOR: If (neither A nor B) then C



NOT: If (not A) then B

From “Cause-Effect Graphs” to “Decision Tables”

Car Insurance Renewal Example

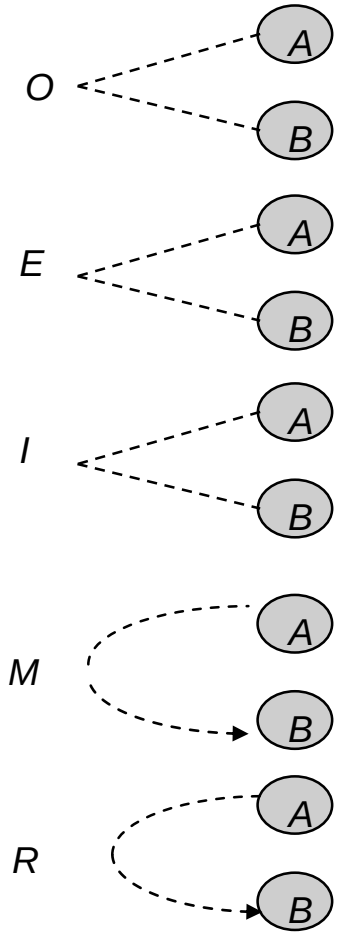


Identical

	Condition Section		Action Section		
Variant	**Claims**	**Age**	**Premium Increase $**	**Send Warning**	**Cancel**
1	**0**	**25-**	**50**	**No**	**No**
2	**0**	**26+**	**25**	**No**	**No**
3	**1**	**25-**	**100**	**Yes**	**No**
4	**1**	**26+**	**50**	**No**	**No**
5	**2 to 4**	**25-**	**400**	**Yes**	**No**
6	**2 to 4**	**26+**	**200**	**Yes**	**No**
7	**5+**	**Any**	**0**	**No**	**Yes**

Cause-Effect Graphs

Additional Constraints



Exactly one of A and B must be true

At most one of A and B may be true
(E should be remembered!)

At least one of A and B must be true

A masks B, i.e., A => NOT B

A requires B, i.e., A => B

Motivations: They are key to minimize test cases (logic minimization), but one must be very careful. They need to be reviewed carefully.

Discussion

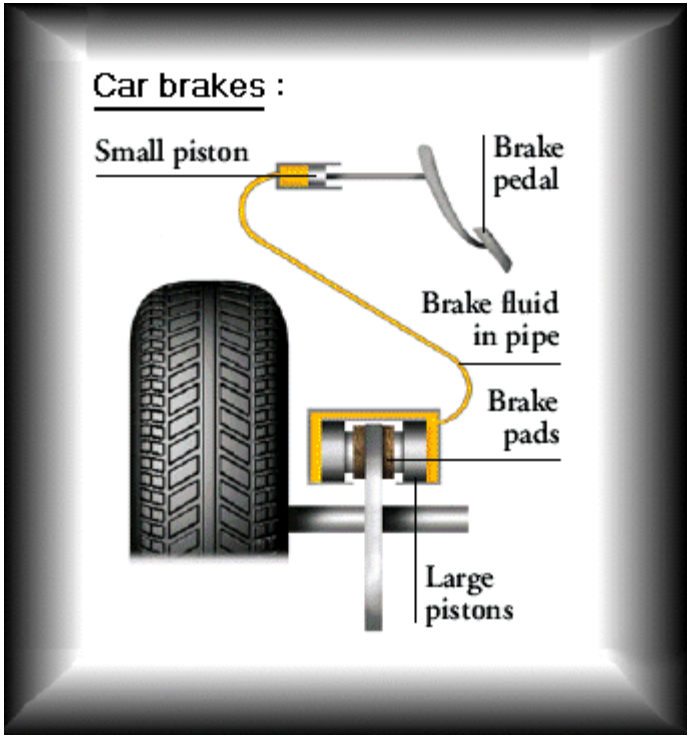
- The cause-effect graph can be used to generate all *possible* combinations of causes and checking whether the effect corresponds to the specification
- It provides a test oracle and specifies constraints on outputs (effects), helping detecting wrong system states and action combinations
- If the graph is too large, for each admissible combination of effects, we should find some combinations of causes that cause that combination of effects by tracing back through the graph
- Because of additional constraints on a cause-effect graph, it can be more restrictive than straight decision tables

Basic Testing Definitions: a non-Software Example

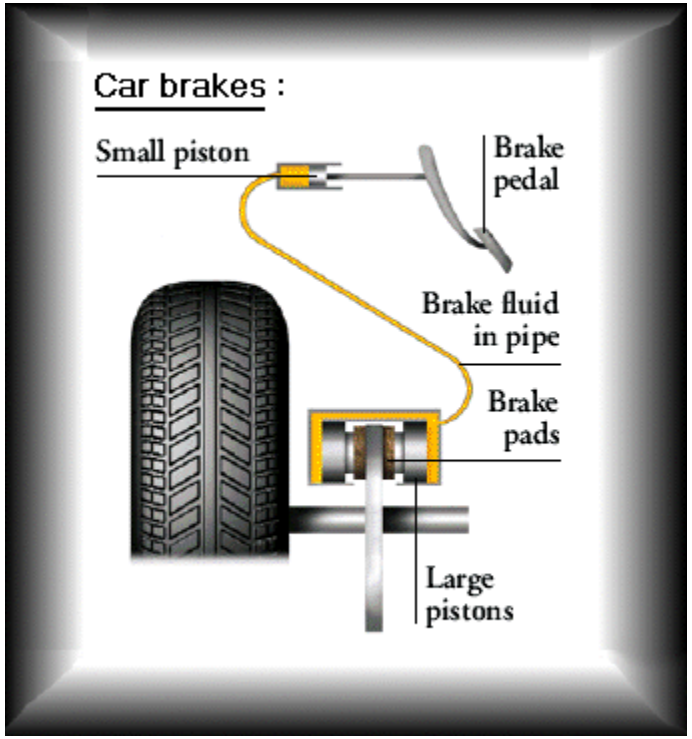
IEEE TRANSACTIONS ON RELIABILITY, VOL. 46, NO. 4, 1997 DECEMBER

Defect, Fault, Error, . . . , or Failure?

Behrooz Parhami, *Fellow IEEE*
University of California, Santa Barabara



- An automobile brake system with a weak joint in the brake fluid piping (e.g., caused by a design flaw) is defective.
- Error: design flaw (human error)
- (Potential) Fault: weak joint in the piping
- A careful testing or inspection activity on the automobile can reveal the fault.
- However, the driver does not automatically notice the fault while driving.

Basic Testing Definitions: a non-Software Example

- A failure happens if either during normal driving or a testing activity, the fault reveals itself.
- If the weak-joint breaks down, the brake system then have a failure.
- It could be that the weak-joint breaks down in one break and the other 3 breaks work, the driver or tester may not notice the fault. This will be a hidden fault. A failure has happened, but it is also hidden.
- Incident: the break system fails and an accident happens.